Libro di Testo — Informatica Classe 2

Corso di Informatica — a.s. 2026/27

Versione 1.0 · 29/10/2026

Corso a cura del prof. Nicola Regge

0 Come lavoriamo (leggere per prime)

1. Si lavora un passo alla volta, con un risultato concreto da vedere subito.
2. Consegne e materiali stanno su Google Classroom; i progetti stanno su GitHub, ognuno nel suo repository.
3. Ogni lezione ha la sua pagina con le schede di teoria, il ripasso e la verifica: leggi le schede prima di rispondere.
4. Se ti blocchi, chiedi all'assistente nella casella della pagina: risponde subito.
5. L'AI aiuta a capire, non a saltare il pensiero: la prova che hai capito è saperlo spiegare a voce con parole tue.
6. Prima del libro c'era la dispensa dei tipi di file (dispensa-corso-classe2, v0.1): il suo contenuto ora è la lezione del 28/09.

1 Le lezioni una per una

In ordine di data: che cosa abbiamo fatto, la teoria e l'esercitazione di ogni lezione.

L01 21/09/2026 — Benvenuti e GitHub

Libro parziale: lezioni/2026-09-21_benvenuti-github.md

Che cosa abbiamo fatto

1. **Quando:** 21/09/2026, 3 ore, Laboratorio.
2. **Argomenti:** Presentazione dell'anno; GitHub: che cos'è, a che cosa serve, l'account.

La teoria e l'esercitazione

Questa lezione è stata di organizzazione e di avvio: non ha una dispensa scritta. Gli argomenti sono quelli elencati sopra.

L02 22/09/2026 — Controllo di account e utenze

Libro parziale: lezioni/2026-09-22_controllo-account.md

Che cosa abbiamo fatto

1. **Quando:** 22/09/2026, 1 ora, Sicurezza.

2. **Argomenti:** Controllo che ognuno entri nei suoi account e nelle sue aree di lavoro.

La teoria e l’esercitazione

Questa lezione è stata di organizzazione e di avvio: non ha una dispensa scritta. Gli argomenti sono quelli elencati sopra.

L03 25/09/2026 — Primo lavoro su GitHub

Libro parziale: lezioni/2026-09-25_primo-lavoro-github.md

Che cosa abbiamo fatto

1. **Quando:** 25/09/2026, 2 ore, Informatica.
2. **Argomenti:** Creazione dell’account Git; primo lavoro su GitHub: repository, file, commit e versioni.

La teoria e l’esercitazione

Dispensa · Classe 2 · Informatica · v2.2

GitHub e il versioning

quello che abbiamo scritto insieme alla lavagna, spiegato per bene

Partiamo dallo schema della **nostra lavagna** e lo apriamo pezzo per pezzo: cos’è GitHub, che tipo di servizio è (PaaS, Freemium) e come funzionano le **versioni**. Con esempi e senza comandi da digitare. Classe 2 · Informatica.

1. Partiamo dalla nostra lavagna

Ecco, ridisegnato in ordine, lo schema che abbiamo costruito insieme alla lavagna. Ogni riquadro è una tappa: nelle prossime pagine le apriamo una per una.

GitHub

è un’AZIENDA è un SERVIZIO

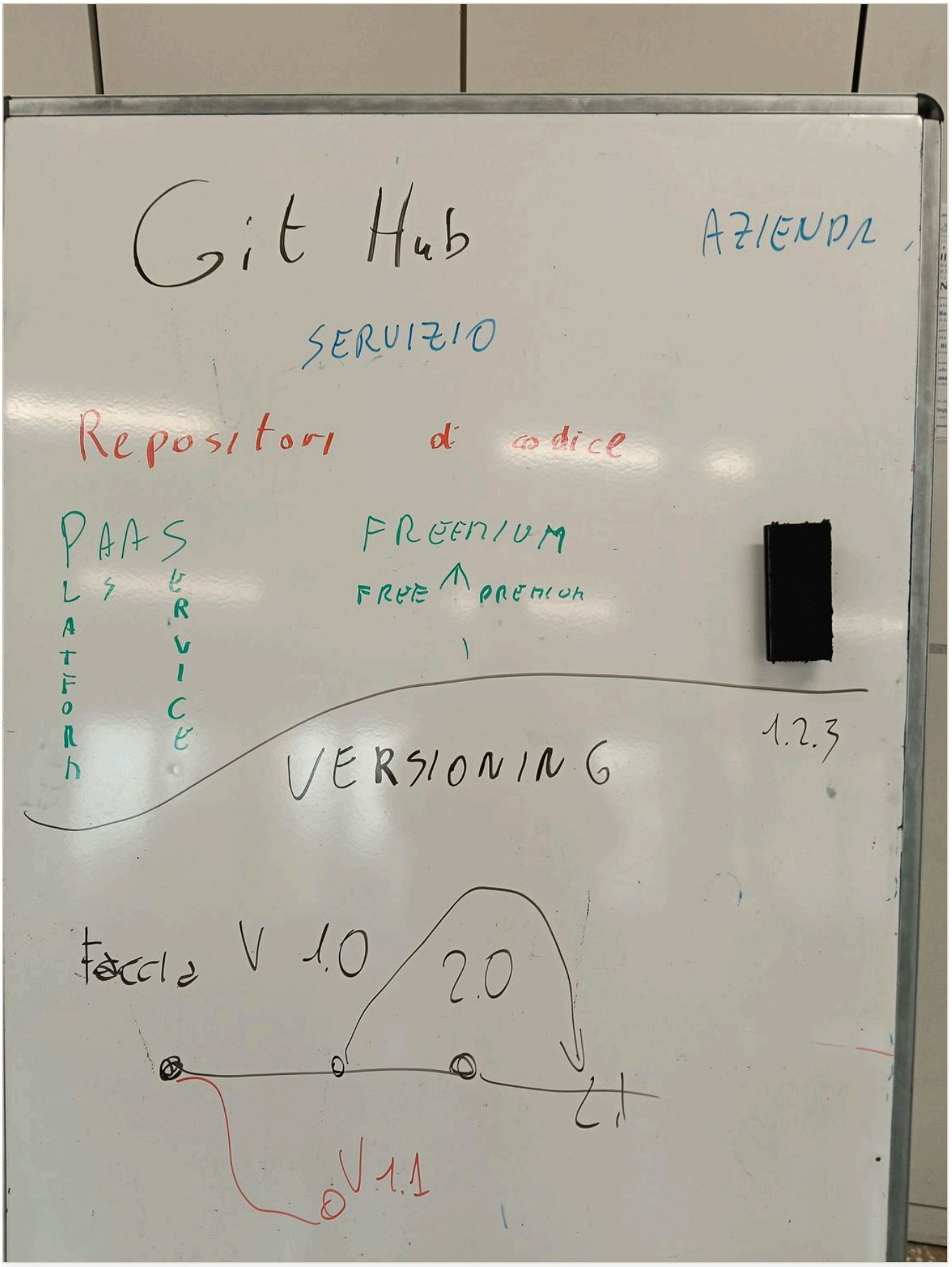
tiene Repository di codice

PaaSPlatform as a Service FREEMIUMfree premium

VERSIONING le versioni: 1 . 2 . 3

v1.0v1.1v2.0 le versioni pubblicate crescono nel tempo

Lo schema della lezione: GitHub è azienda e servizio; tiene repository di codice; come servizio è una piattaforma (PaaS) col modello Freemium; e il progetto cresce per versioni (versioning).



La **lavagna della lezione** (3ª ora): lo schema originale scritto in classe, da cui parte questa dispensa.

2. GitHub: un’azienda e un servizio

Alla lavagna abbiamo scritto due parole sopra “GitHub”: **azienda** e **servizio**. Vediamo perché tutte e due sono vere.

È un’azienda

Dietro GitHub c’è una **società vera** — **GitHub, Inc.**, nata nel 2008 e dal 2018 di proprietà di **Microsoft**. Come ogni azienda deve mantenersi: ci arriviamo al punto “Freemium”.

È un servizio (online)

GitHub è un **sito** dove metti i tuoi progetti **online**: per tenerli al sicuro, mostrarli e lavorarci in gruppo. Non è un programma che installi col CD: lo usi dal **browser**.

Attenzione a non confondere: Git è lo **strumento** (la tecnologia che tiene la storia di un progetto, gratis e di nessuno); **GitHub** è il **servizio online** (l’azienda) dove quei progetti vivono in rete. Git è il **motore**, GitHub è il **garage online**.

3. Che cosa tiene: i repository di codice

Sulla lavagna, sotto “servizio”, abbiamo scritto **“repository di codice”**: è ciò che GitHub custodisce.

Un **repository** (in breve “repo”) è la **cartella-progetto** con dentro i file **e tutta la loro storia**. Ogni salvataggio della storia si chiama **commit** (una “fotografia” del progetto con un messaggio). Se lavori in gruppo, puoi aprire un **branch** (una strada di lavoro parallela) e poi riunirlo con un **merge**.

Esempio. Il tuo gioco “SpaceRun” è un repository su GitHub: dentro c’è il gioco **e** il diario di tutte le modifiche. Puoi tornare a com’era la settimana scorsa in qualsiasi momento.

4. Che tipo di servizio è: PaaS

Alla lavagna: **PaaS = Platform as a Service**, cioè “piattaforma **come servizio**”. Vuol dire che qualcuno ti mette a disposizione online una **piattaforma già pronta**, e tu la usi senza costruirti e mantenere i computer, i server e tutto il resto.

Analogia della cucina. Per aprire un ristorante puoi **costruirti la cucina da zero** (tanta fatica) oppure **affittare una cucina già attrezzata** e pensare solo a

cucinare. GitHub è la cucina già pronta: tu porti il progetto, il resto lo mette lui.

La famiglia "...aaS"

PaaS non è da solo: fa parte di una famiglia di servizi online, tutti con la sigla che finisce in **"aaS"** (*as a Service*). Cambia quanto lavoro fai tu:

Sigla	Nome	Cosa ti danno pronto	Esempio
IaaS	Infrastructure as a Service	Solo i "muri e la corrente": server nudi.	Amazon AWS, Google Cloud
PaaS	Platform as a Service	La cucina già attrezzata: piattaforma pronta.	GitHub, Netlify
SaaS	Software as a Service	Il piatto già servito: un programma pronto nel browser.	Gmail, Google Documenti

Come ricordarli — la pizzeria. IaaS = affitti il locale vuoto. **PaaS** = affitti la cucina attrezzata. **SaaS** = ti fai consegnare la pizza pronta. Più scendi, meno fatica fai.

5. Come si paga: il modello Freemium

Sulla lavagna: **Freemium = FREE PREMIUM**. È una parola nata unendo **free** (gratis) e **premium** (a pagamento). Significa: le funzioni **di base sono gratis** per tutti; se vuoi funzioni **in più**, paghi un abbonamento.

Esempio che conosci. Tanti giochi per telefono e app come Spotify funzionano così: gratis con qualche limite, a pagamento per togliere pubblicità o avere funzioni extra. **GitHub è uguale:** a te studente basta e avanza la parte gratis.

Ecco perché "azienda" e "Freemium" stanno insieme: GitHub è un'azienda, quindi deve guadagnare; lo fa lasciando gratis l'uso normale e facendo pagare le funzioni avanzate a chi ne ha bisogno (aziende, team grandi).

6. Il versioning: le versioni 1.2.3

L'ultima parte della lavagna: **VERSIONING**, con il numero 1.2.3. Quando un progetto raggiunge un punto "buono", gli si dà un **numero di versione** fatto di tre cifre:

Cifra	Nome	Aumenta quando…
1	major	cambi **tutto**, in modo grosso (magari incompatibile col vecchio)
2	minor	aggiungi una **funzione nuova**, ma il resto funziona ancora
3	patch	correggi solo un **bug**, niente di nuovo

Esempio. v1.0.0 prima versione giocabile ⟶ aggiungi i suoni: v1.1.0 ⟶ correggi un bug: v1.1.1 ⟶ rifai tutto da capo: v2.0.0.

Il grafo che abbiamo disegnato

Alla lavagna avevamo disegnato la linea delle versioni con un ramo che si stacca e rientra. Ecco cosa racconta: il progetto va avanti su main, ogni tanto si pubblica una versione (le bandierine), e un ramo di lavoro si può staccare e poi riunire.

v1.0v2.0

v1.1 (ramo)

Da v1.0 il progetto cresce; un ramo porta la v1.1 e si riunisce; poi un grande cambiamento porta alla v2.0.

Regola d'oro: una versione già pubblicata **non si riusa mai**. Se cambi anche una sola cosa, prima **aumenti** il numero e poi pubblichi.

7. Casi d'uso: 15 grafi con spiegazione (fino a 5 livelli)

Vediamo Git/versioning "in azione". In ogni grafo il **pallino** è un commit/versione e la **linea** è il tempo (da sinistra a destra). I primi sono semplici; gli ultimi arrivano a **cinque livelli** di rami.

Come si legge un grafo: ogni **pallino** è un **commit** (una fotografia salvata). La **linea orizzontale** in alto è di solito main, il ramo principale. Le linee colorate che si staccano sono **branch** (rami di lavoro). Quando due linee si riuniscono è un **merge**. Le **bandierine gialle** sono le **release** (versioni pubblicate, es. v1.0).

1. Sviluppo lineare (da soli, tutto dritto)

Il caso più semplice: un solo autore, nessun ramo. Ogni commit segue il precedente in fila. La storia è una linea dritta. Va benissimo per progetti piccoli.

avviopaginastiletestifixfine

2. Una funzione su un branch, poi merge

Provi una funzione nuova su un ramo separato, così il main resta sempre funzionante. Quando è pronta, la **unisci** (merge) al main.

main

funzione “login” merge

3. Due funzioni in parallelo, merge in tempi diversi

Due rami partono insieme ma finiscono in momenti diversi: uno si fonde prima, l’altro dopo. Il main raccoglie i pezzi man mano che sono pronti.

main

grafica

suoni (finisce dopo)

4. Hotfix urgente durante una funzione lunga

Stai lavorando a una funzione che richiede giorni, ma esce un **bug grave** in produzione. Apri un ramo **hotfix** corto, lo correggi e lo unisci subito; poi torni alla tua funzione.

main

hotfix (bug)

la mia funzione lunga (continua)

5. Le release pubblicate (bandierine sul main)

Man mano che il progetto avanza, si **pubblicano** le versioni: v1.0 giocabile, poi correzioni e aggiunte fino a un cambiamento grosso v2.0. Ogni bandierina è una versione “congelata”.

v1.0v1.1v1.2v1.3v2.0 +suoni+livellifixtutto nuovo

6. Un ramo abbandonato (idea scartata)

Non tutte le idee funzionano. Qui un ramo viene provato ma **mai unito**: si decide che non va e lo si lascia lì. Il bello di Git: il main non ne risente per niente.

main

idea “grafica 3D” — abbandonata

7. Un ramo che nasce da un altro ramo

A volte una funzione è così grande che, mentre la sviluppi, ne stacchi un pezzo su un **sotto-ramo**. Prima rientra il sotto-ramo nel ramo, poi il ramo nel main. Rami dentro rami.

main

funzione “mappa”

sotto-ramo “zoom”

8. Grande integrazione (progetto di gruppo)

Nel lavoro di squadra ognuno ha il suo ramo (grafica, movimenti, audio, menu). Alla fine confluiscono tutti nel main con le loro Pull Request: nasce la versione completa.

main grafica movimenti audio menu v1.0

9. Ramo di manutenzione a lungo termine

Esci con la v1.0 e il main va avanti verso la v2.0 (tutto nuovo). Ma chi usa ancora la v1 ha bisogno di correzioni: si tiene un **ramo di manutenzione** parallelo dove escono v1.1, v1.2 solo con i fix.

main v2.0 v1.0

ramo manutenzione v1.x v1.1v1.2

10. Revert: annullare una modifica sbagliata

Un commit ha introdotto un problema. Invece di cancellare la storia (non si fa), si aggiunge un **nuovo commit** che **annulla** quello sbagliato: è il **revert**. La storia resta onesta e il progetto torna a posto.

main commit sbagliato revert (annulla)

11. Conflitto di merge risolto

Due rami hanno cambiato **la stessa riga** in modo diverso. Quando si uniscono, Git non sa quale tenere: è un **conflitto**. Una persona decide cosa tenere e chiude il merge con un commit speciale di “risoluzione”.

main cambio A cambio B (stessa riga)

merge risolto

12. Squadra a 5 rami con due release

Un progetto vero di gruppo: sul **main** si pubblicano le versioni, mentre **quattro rami** diversi (grafica, movimenti, audio, menu) lavorano in parallelo e rientrano in momenti diversi. Cinque linee in tutto.

main

v1.0v2.0

grafica

movimenti

audio

menu

13. Rami annidati: cinque livelli di profondità

Una funzione grande si divide in pezzi: dal main nasce un ramo, da quel ramo un **sotto-ramo**, e ancora più giù. Ogni livello rientra nel precedente. Qui si scende di **cinque livelli**.

main livello 1 livello 2 livello 3 livello 4

14. Due versioni major mantenute insieme

Dopo la v1.0 il main punta alla v2.0, ma la v1 resta in uso: un **ramo di manutenzione** sforna v1.1/v1.2 (solo fix), e più tardi anche la v2 avrà il suo ramo di manutenzione. Tante linee vive insieme.

main v1.0v2.0

v1.1v1.2v1.3ramo manutenzione v1.x

v2.1ramo manutenzione v2.x

15. Storia realistica: rami, hotfix, revert e conflitto

Il quadro completo: il main avanza con le release; una **feature** lunga con un **sotto-ramo**; un **hotfix** urgente; un **revert** per annullare un errore; e un **merge con**

conflitto risolto. Cinque livelli di attività insieme.

main errorerevert v2.0

hotfix

feature sotto-ramo

ramo parallelo (stessa riga) ⟶ merge con conflitto

In sintesi: con pochi elementi — commit, branch, merge, tag/release — si descrivono **tutte** le situazioni di un progetto reale, dalla più semplice (linea dritta) alle più complesse (integrazione di squadra, manutenzione parallela, conflitti). Saper **leggere** questi grafi è già metà del lavoro.

8. Riepilogo (la lavagna in tre righe)

1. **GitHub** è un'**azienda** e un **servizio** online dove tieni i tuoi **repository di codice** (con Git sotto, come motore).
2. Come servizio è una **piattaforma pronta (PaaS)**, della famiglia **…aaS**, e si paga in **Freemium** (base gratis, extra a pagamento).
3. Il progetto cresce per **versioni** (1.2.3): major, minor, patch — e una versione pubblicata non si tocca più.

Per vedere tante situazioni diverse con i grafi, guarda la scheda "Versioning — 11 casi d'uso".

9. Glossario

Parola	Vuol dire
GitHub	il sito-servizio (azienda di Microsoft) che ospita i progetti online
Git	lo strumento (gratis) che tiene la storia di un progetto — il “motore” sotto GitHub
repository	la cartella-progetto con dentro i file e tutta la loro storia
commit	un salvataggio (una fotografia) del progetto con un messaggio
branch	una strada di lavoro parallela, per provare senza rovinare il main
merge	unire un ramo con quello principale (main)
PaaS	“piattaforma come servizio”: una piattaforma online già pronta (es. GitHub)
IaaS / SaaS	gli altri servizi …aaS: infrastruttura (server nudi) / software pronto nel browser
Freemium	base gratis, funzioni avanzate a pagamento
versioning	dare un numero (v1.2.3) alle versioni: major . minor . patch
release	una versione pubblicata e “congelata”; non si riusa mai un numero già dato

L04 28/09/2026 — I tipi di file

Libro parziale: lezioni/2026-09-28_tipi-di-file.md

Che cosa abbiamo fatto

1. **Quando:** 28/09/2026, 3 ore, Laboratorio.
2. **Argomenti:** Che cos’è un file, l’estensione, file di testo e file binari, il dizionario dei formati, la compressione; compito sui tipi di file.

La teoria e l’esercitazione

Dispensa · Classe 2 · Informatica · v1.1

I tipi di file

a cosa servono, gli esempi, i file “a tag”, video e compressione

Ogni file ha un **tipo** (lo dice l’estensione: .md, .mp4…). Capire i tipi serve a sapere **cosa contiene** un file, **con che programma** si apre e **perché** pesa tanto o poco. Con esempi da mondi diversi.

Di cosa parliamo

1. Cos’è un file e a cosa serve l’estensione **2.** Le grandi famiglie di file (testo, immagini, audio, video, archivi…) **3.** Dizionario dei formati (uno per uno, con esempio) **4.** I file “a TAG”: HTML, XML, MD — con esempi veri **5.** La compressione: perché un file diventa più piccolo **6.** Il Markdown (MD) e l’intelligenza artificiale **7.** Glossario

1. Cos’è un file e a cosa serve l’estensione

Un **file** è un contenitore di informazioni salvato nel computer: un testo, una foto, una canzone, un video, un programma. Ogni file ha un **nome** e, alla fine, un’**estensione**: le lettere dopo il punto (es. canzone.mp3, foto.jpg).

L’estensione è come il “cognome” del file: dice di che **tipo** è, così il computer sa **con quale programma** aprirlo. Cambiare l’estensione non cambia il contenuto: rinominare foto.jpg in foto.txt non la trasforma in testo, la fa solo aprire con il programma sbagliato.

Esempio quotidiano. È come le **etichette** sui barattoli in cucina: “zucchero”, “sale”. Il contenuto è dentro; l’etichetta ti dice cosa aspettarti e come usarlo.

Il percorso (path): dove si trova il file

Oltre al tipo, un file ha un **indirizzo** nel computer: il **percorso** (in inglese **path**), cioè la lista di cartelle per arrivarci, es. C:\Documenti\Scuola\appunti.txt. Su Windows le cartelle si separano con la barra rovescia \ — è un’eredità del vecchio sistema **DOS**, dove tutto si scriveva a mano (es. C:\DOS).

2. Due grandi mondi: TESTO e BINARIO

La prima grande divisione (come alla lavagna) è tra file di **testo** e file **binari**.

> **1. File di TESTO (TXT):** dentro ci sono solo **lettere e simboli**, quindi li puoi aprire col **Blocco note** e **leggerli**. Sono a “formato libero”. Esempi: txt, md, html, css, js, xml, csv, json, .gd.
>
> **2. File BINARI (BIN):** dentro ci sono **codici** che il Blocco note non sa mostrare (escono simboli strani): servono **programmi specifici**. Esempi: doc, xls, pdf, dwg, png, jpeg, zip, exe, mp3, mp4, wav.

Prova del nove. Per sapere se un file è testo o binario, aprilo col **Blocco note**: se lo **leggi** è testo; se vedi **simboli strani** è binario.

Le famiglie più in dettaglio

Dentro questi due mondi ci sono le **famiglie** per uso:

Famiglia	A cosa serve	Esempi di estensione
Testo semplice	solo lettere, nessuna formattazione	.txt .md
Testo "a tag"	testo con etichette che danno struttura	.html .xml
Dati / tabelle	elenchi e tabelle di dati	.csv .json
Documenti impaginati	testo formattato, pronto da leggere/stampare	.pdf .docx .xlsx
Immagini	foto e disegni	.jpg .png
Audio / Video	suoni e filmati	.mp3 .mp4 .wav
Archivi	tanti file compressi in uno solo	.zip
Pagine web	struttura, stile e comportamento dei siti	.html .css .js
Disegno tecnico (CAD)	disegni e progetti tecnici	.dwg
Programmi (eseguibili)	si avviano col doppio clic	.exe
Codice / progetto	istruzioni per un programma	.gd (Godot) .py

3. Dizionario dei formati (uno per uno)

Per ognuno: **a cosa serve**, un **esempio**, e **con che programma si apre** (a scuola: Blocco note, browser, Godot portabile, SumatraPDF).

TXT — testo semplice

Solo testo, senza grassetti né colori. Leggerissimo. Si apre col **Blocco note**. Esempio: una lista della spesa, degli appunti veloci.

MD (Markdown) — testo con formattazione "leggera"

Testo semplice ma con simboli che diventano formattazione: # = titolo, **…** = grassetto. È il formato con cui scriviamo le dispense (poi diventano PDF). Si legge su GitHub o col Blocco note.

HTML — la pagina web

Il linguaggio delle **pagine dei siti**. Usa i **tag** (vedi §4). Lo apre il **browser**, che trasforma i tag in una pagina bella da vedere.

XML — dati con etichette

Come HTML usa i tag, ma per **organizzare dati** (non per fare pagine). Tanti programmi si scambiano informazioni in XML.

CSV — tabella semplice

Una tabella scritta come testo: ogni riga è una riga della tabella, le colonne separate da **virgole** (comma). Lo aprono Excel/Fogli Google. Esempio: l'esportazione dei voti di un quiz.

JSON — dati per i programmi

Come CSV ma per dati più complessi (a "scatole dentro scatole"). È il modo con cui le app si scambiano dati su Internet.

PDF — il documento "congelato"

Un documento impaginato che si vede **uguale ovunque** e si stampa bene, ma non si modifica facilmente. Si apre con **SumatraPDF** o il browser. Le nostre dispense finali sono PDF.

DOCX / XLSX — Word ed Excel

Documento di testo (DOCX) e foglio di calcolo (XLSX). Servono per **scrivere e modificare**. Li aprono Word/Excel o Documenti/Fogli Google.

JPG / PNG — le immagini

JPG: foto, molto compresse (leggere, ma perdono un po' di qualità). **PNG**: disegni/loghi, senza perdita e con sfondo trasparente. Le apre qualsiasi visualizzatore o il browser.

MP3 / MP4 — audio e video

MP3 = audio compresso. **MP4** = video (immagini in movimento + audio insieme). Li apre il lettore multimediale o il browser.

ZIP — l'archivio

Una **valigia**: mette tanti file dentro uno solo e li **comprime** per pesare meno e spedirli comodi. Si "estrae" per riavere i file dentro.

CSS / JS — stile e comportamento delle pagine

Vanno insieme all'HTML: **CSS** dà lo **stile** (colori, caratteri, posizioni), **JS** (JavaScript) dà il **comportamento** (bottoni che fanno cose). Sono testo, li apre il Blocco note; il browser li usa con l'HTML.

DWG — il disegno tecnico (CAD)

Il formato dei **disegni tecnici** fatti con programmi CAD (es. AutoCAD): piantine, progetti. È binario, serve il programma giusto per aprirlo.

EXE — il programma da avviare

Un **programma eseguibile**: col doppio clic **parte**. Godot portabile è un .exe. Attenzione: aprire un .exe che non conosci è rischioso (può contenere virus).

WAV — audio senza compressione

Audio di qualità piena ma **pesante** perché **non compresso** (a differenza dell'MP3). Si usa quando serve la massima qualità.

GD — lo script di Godot

Un file di **codice**: le istruzioni del nostro gioco in Godot. È testo, ma lo apre **Godot** (col doppio clic parte il gioco: per leggerlo come testo, tasto destro ⟶ Apri con ⟶ Blocco note).

4. I file "a TAG": HTML, XML, MD

Alcuni file di testo usano i **tag** (etichette). Un **tag** è un'etichetta tra parentesi angolari che **marca** un pezzo di testo dicendo "questo è un titolo", "questo è grassetto". Di solito vanno a coppie: uno apre <titolo> e uno chiude </titolo>.

Analogia. I tag sono come gli **evidenziatori e le note a margine** su un foglio: il testo è lo stesso, ma le marcature dicono a cosa serve ogni parte.

Esempio HTML (pagina web)

<h1>La mia prima pagina</h1> Ciao, questo è un paragrafo in grassetto. vai a GitHub

<h1> = titolo grande · = paragrafo · = grassetto · <a> = link. Il browser legge questi tag e disegna la pagina.

Esempio XML (dati)

<studente> <nome>Mario</nome> <classe>3 INF</classe> <voto>90</voto> </studente>

Qui i tag non servono a fare una pagina, ma a **organizzare i dati**: si capisce quale pezzo è il nome, quale la classe, quale il voto.

Esempio MD (Markdown)

Titolo grande

Sottotitolo

Testo normale con una parola in **grassetto**.

1. primo punto
2. secondo punto

Il Markdown non usa parentesi angolari ma **simboli leggeri** (#, **, -): più veloce da scrivere a mano, poi un programma lo trasforma in PDF o pagina.

In comune: tutti e tre sono **testo** (li apre anche il Blocco note), ma con **marcature** che danno **struttura**. Cambia solo il “dialetto”: angolari per HTML/XML, simboli per MD.

5. La compressione: perché un file diventa più piccolo

Comprimere vuol dire scrivere la stessa informazione occupando **meno spazio**. Serve a far pesare meno i file (per salvarli, inviarli, metterli online). Ci sono due modi:

1. **Senza perdita (lossless):** si comprime ma si può tornare **identici** all'originale. È come **piegare bene** un vestito nella valigia: occupa meno, ma è lo stesso vestito. Esempi: .zip, .png.
2. **Con perdita (lossy):** si butta via un po' di dettaglio che quasi non si nota, per pesare **molto** meno. È come **riassumere** un tema: perdi qualche parola ma il senso resta. Esempi: .jpg, .mp3, .mp4.

Esempio concreto. La stessa foto: in **PNG** senza perdita può pesare 8 MB; in **JPG** con perdita 0,8 MB — dieci volte meno, e a occhio quasi uguale. Per una foto sul telefono va benissimo il JPG; per un logo con linee nette meglio il PNG.

Perché ci riguarda: un video lungo non compresso peserebbe **giga e giga**; grazie all'MP4 (compressione con perdita) sta in pochi MB e si guarda in streaming. È lo stesso motivo per cui abbiamo **compresso l'immagine** nelle dispense: pesare meno senza perdere qualità utile.

6. Il Markdown (MD) e l'intelligenza artificiale

C'è un motivo se vi diamo le dispense anche in **MD** (Markdown), oltre che in PDF: il Markdown è il formato **perfetto da dare a un'intelligenza artificiale** (come Gemini o ChatGPT) per farsi **spiegare** o **tradurre** un testo.

Perché l'AI legge bene un MD

1. **È testo puro:** l'AI legge **esattamente** il contenuto, senza la "sporcizia" nascosta di un PDF o di un DOCX (che dentro hanno codici di formattazione, e da binari sono illeggibili).
2. **La struttura è chiara:** i simboli del Markdown (# titolo, ** grassetto, - elenco) dicono all'AI **com'è organizzato** il testo, così capisce cosa è titolo, cosa è elenco, cosa è importante.
3. **È leggero:** si **copia e incolla** facilmente dentro una chat con l'AI.

Analogia. Dare all'AI un MD è come darle un foglio **pulito e ordinato** invece di uno pieno di scarabocchi e formattazioni strane: lo capisce molto meglio.

Come lo usate voi: prendete il file .md della dispensa, lo **date alla vostra AI** e le chiedete, ad esempio: "*spiegamelo in modo semplice*" oppure "*traducilo nella mia lingua*". Così ognuno ripassa la lezione **come e nella lingua che preferisce**.

Regola d'uso dell'AI: serve per **capire meglio**, non per saltare il ragionamento. La prova che hai capito è **saperlo spiegare a voce, con parole tue**.

7. Glossario

Parola	Vuol dire
file	un contenitore di informazioni salvato nel computer
percorso (path)	l'indirizzo del file nel computer (es. C:\cartella\file.txt)
file di testo	leggibile col Blocco note (txt, md, html, css, js, xml, csv…)
file binario	serve un programma specifico (doc, pdf, png, mp4, exe, zip…)
estensione	le lettere dopo il punto: dicono il tipo del file (.mp4, .md…)
tag	un'etichetta (es. <titolo>) che marca un pezzo di testo
HTML	il linguaggio a tag delle pagine web
XML	testo a tag per organizzare dati
Markdown (MD)	testo con simboli leggeri per la formattazione
CSV	tabella scritta come testo, colonne separate da virgole
MP4	formato video (immagini in movimento + audio, compressi)
compressione	scrivere la stessa informazione occupando meno spazio
lossless / lossy	senza perdita (identico) / con perdita (più leggero)
ZIP	archivio: tanti file compressi in uno solo

L05 29/09/2026 — Lazarus: convertitore mph e km/h

Libro parziale: lezioni/2026-09-29_convertitore-mph-kmh.md

Che cosa abbiamo fatto

1. **Quando:** 29/09/2026, 2 ore, Informatica.
2. **Argomenti:** Programma di conversione tra miglia orarie e chilometri orari in Lazarus: TEdit, RadioGroup, StrToFloat e FormatFloat.

La teoria e l'esercitazione

*Questo file è **per te**: puoi leggerlo e puoi **darlo a Gemini** (o a un'altra AI) per farti spiegare le parti che non capisci, o farti tradurre nella tua lingua. Regola d'oro: l'AI serve per **capire meglio**, non per saltare il pensiero. Alla fine devi saper **spiegare a voce** cosa fa il tuo programma.*

1. Cosa costruiamo

Un'app che converte una velocità tra **km/h** e **mph** (miglia all'ora). Scrivi un numero, scegli la direzione con dei pallini, premi **Converti** e leggi il risultato con **una cifra decimale**.

2. UI — l'interfaccia (cosa si vede)

1. **Edit1** con etichetta **Km/h** e **Edit2** con etichetta **mph** (le due caselle).
2. **RadioGroup1** (pallini della direzione). Per inserire le due voci: nell'Object Inspector trova la proprietà **Items**, clicca il pulsante `…` a destra e nell'editor scrivi **una voce per riga**: `Da Km/h` e `Da mph`, poi OK. Infine metti `ItemIndex = 0`.
3. **Button1** con `Caption = Converti`.

3. UX — come si usa (con mouse E con sola tastiera)

L'app gira su **Windows** e si deve poter usare in due modi: col **mouse** e anche **solo con la tastiera**. Si regola con le proprietà dei controlli:

1. **TabStop** (True/False): se True il controllo si raggiunge con `Tab`; se False viene saltato.
2. **TabOrder**: l'ordine con cui `Tab` passa da un controllo all'altro (0, 1, 2…). Ordine logico: Edit1 ⟶ RadioGroup ⟶ Edit2 ⟶ Button.
3. **Button1.Default = True**: premendo `Invio` parte la conversione senza cliccare.

4. LOGICA — il calcolo

1. **1 mph ≈ 1,6 km/h** (in classe usiamo 1,6; il valore preciso è 1,60934).
2. Da **Km/h** a mph: si **divide** per 1,6. Da **mph** a Km/h: si **moltiplica** per 1,6.

Il codice del bottone:

```
procedure TForm1.Button1Click(Sender: TObject);
var
  a: Real;
begin
```

```
  if RadioGroup1.ItemIndex = 0 then      // Da Km/h a mph
  begin
    a := StrToFloat(Edit1.Text);         // Edit1 = Km/h
    Edit2.Text := FormatFloat('0.0', a / 1.6);
  end
  else                                    // Da mph a Km/h
  begin
    a := StrToFloat(Edit2.Text);         // Edit2 = mph
    Edit1.Text := FormatFloat('0.0', a * 1.6);
  end;
end;
```

1. `StrToFloat(...)` trasforma il **testo** della casella in un **numero** con la virgola.
2. `FormatFloat('0.0', ...)` trasforma il numero in **testo con 1 cifra decimale**.

5. Come chiedere aiuto a Gemini (esempi di domande)

Copia questo file dentro Gemini e prova a chiedere, con parole tue:

1. «Spiegami con parole semplici cosa fa `StrToFloat` in questo codice.»
2. «Perché serve `FormatFloat('0.0', ...)`? Cosa cambia se scrivo `'0.00'`?»
3. «Cos'è `ItemIndex` del RadioGroup e perché il primo pallino è 0?»
4. «Traducimi questa scheda in [la tua lingua], così la capisco meglio.»
5. «Fammi 3 domande per controllare se ho capito il convertitore.»

> *Ricorda: dopo che l'AI ti ha spiegato, **prova a rifarlo tu** e a spiegarlo a voce a un compagno. Se lo sai spiegare, l'hai capito davvero.*

L06 01/10/2026 — Convertitore di temperature e Sicurezza 1

Libro parziale: lezioni/2026-10-01_convertitore-temperature-sicurezza.md

Che cosa abbiamo fatto

1. **Quando:** 01/10/2026, 4 ore, Laboratorio e Sicurezza.
2. **Argomenti:** Lazarus «Convertitore di temperature» (Celsius e Fahrenheit): if/then, begin/end annidati, verso automatico, Reset, finestra di scelta; Sicurezza sul lavoro, modulo 1: rischio, danno, prevenzione e protezione, con quiz e test a risposta aperta.

La teoria e l’esercitazione

> *[ATTENZIONE]* ***ATTENZIONE — DIFFERENZA VOLUTA:*** *questo compito* ***NON*** *è sulle miglia/km della lezione, ma su* ***Celsius Fahrenheit****. È fatto* ***apposta*** *così, per verificare che sai applicare lo* ***stesso metodo*** *a una conversione nuova.* ***Non è un errore.***

Cosa devi fare: costruisci un’app in Lazarus **uguale nella struttura** a quella Km/hmph vista a lezione, ma che converte le **temperature** (°C °F). Cambia solo la formula: il metodo è lo stesso.

1. Componenti (UI)

1. **Edit1** (°C) e **Edit2** (°F).
2. **RadioGroup1** con `Da C a F` e `Da F a C` (ItemIndex = 0).
3. **Button1** con `Caption = Converti`.

2. Comportamento (LOGICA)

1. Leggi il numero con **StrToFloat**.
2. Se ItemIndex = 0: **F = C × 9/5 + 32**. Altrimenti: **C = (F − 32) × 5/9**.
3. Scrivi il risultato con **una cifra decimale** con **FormatFloat(‘0.0’, …)**.

Controlli: 100 °C ⟶ 212.0 °F · 32 °F ⟶ 0.0 °C · 37 °C ⟶ 98.6 °F.

3. Uso con mouse E con tastiera (UX)

1. Deve funzionare su **Windows**, usabile col **mouse** e **solo con tastiera**.
2. Sistema **TabStop** e **TabOrder** (ordine: Edit1 ⟶ RadioGroup ⟶ Edit2 ⟶ Button).
3. `Button1.Default = True` così `Invio` lancia la conversione.

4b. Usa Gemini con la scheda MD

1. Apri la scheda **.md** del convertitore in **Gemini** (incolla o carica il file).
2. Chiedigli **3 domande di verifica** sull’app e **rispondi**.
3. Allega **uno screenshot** della chat (o scrivi cosa ti ha chiesto e cosa hai risposto).

4. Consegna

1. 2 screenshot: app funzionante + codice del bottone.
2. Rispondi: cosa fa StrToFloat? cosa fa FormatFloat('0.0', ...)? a cosa serve TabOrder?
3. Consegna su Classroom.

5. Valutazione (100 punti)

Voce	Punti
Componenti giusti (RadioGroup, 2 Edit, Button)	15
StrToFloat corretto	15
Due formule giuste con RadioGroup (ItemIndex)	25
Risultato con 1 decimale (FormatFloat)	15
Uso con tastiera: TabStop/TabOrder	10
Sa spiegare a voce	10
Uso del file MD con Gemini	10

Conta la comprensione: se il metodo è giusto e sai spiegarlo, va bene anche con piccoli errori.

L07 05/10/2026 — Il programma su GitHub e «Indovina il numero»

Libro parziale: lezioni/2026-10-05_github-indovina-numero.md

Che cosa abbiamo fatto

1. **Quando:** 05/10/2026, Informatica.
2. **Argomenti:** Il proprio programma caricato su GitHub; gioco «Indovina il numero» con gli if annidati.

La teoria e l'esercitazione

v1.0

Le decisioni: if / then / else

e gli if uno dentro l’altro

Classe 2 · Informatica · Lazarus · 05/10/2026 · e alla fine il gioco **Indovina il numero**

Cosa impari oggi: a far **decidere** il programma (**if** = se, **then** = allora, **else** = altrimenti) e a mettere un if **dentro** un altro if (si dice **if annidati** : come una scatola dentro una scatola). Alla fine costruisci un **gioco vero** : il computer pensa un numero e tu lo devi indovinare.

1. Ripasso: if / then / else

Si legge come in italiano: **SE** la condizione è vera **ALLORA** fai questo, **ALTRIMENTI** fai quest’altro.

numero pari?

SÌ -> scrivi PARI

NO -> scrivi DISPARI

```
numero := StrToInt(Edit1.Text);
if numero mod 2 = 0 then
  begin
    Label1.Caption := 'PARI';
  end
else
  begin
    Label1.Caption := 'DISPARI';
  end;
```

`mod` = il **resto** della divisione: se diviso 2 il resto è 0, il numero è pari. I confronti: `=` uguale · `<>` diverso · `<` minore · `>` maggiore · `<=` · `>=`.

2. La regola dei rientri (si fa sempre così)

Regola	Perché
if ed **else** sullo **stesso rientro** (stessa colonna)	si vede subito quale else appartiene a quale if
begin ed **end** sullo **stesso rientro** , con lo stesso numero e colore	ogni begin ha il suo end, come una parentesi che si apre e si chiude
tutto quello che sta **dentro** va **un gradino più a destra** (2 spazi)	si vede a colpo d'occhio chi sta dentro a chi
dopo **then** e dopo **else** mettiamo **sempre** begin … end, anche per una riga sola	il codice resta sempre ordinato e puoi aggiungere righe senza errori

L'errore che fanno tutti (anche i professionisti): il **;** prima di **else**. L'**end** che sta subito prima di **else** va **senza** punto e virgola. Il **;** si mette solo all'**end** finale dell'if.

Trucco per non perdere le coppie: appena scrivi **begin** , scrivi subito sotto il suo **end** allo stesso rientro. Poi riempi in mezzo.

3. Un if dentro un altro if (if annidati)

Con un solo if hai **2 strade**. Se dentro l'else metti un altro if, le strade diventano **3**. Esempio: positivo, negativo o zero?

numero > 0 ?

SÌ -> **POSITIVO**

NO ->

numero < 0 ?

SÌ -> **NEGATIVO**

NO -> **ZERO**

```
numero := StrToInt(Edit1.Text);
if numero > 0 then
  begin
    Label1.Caption := 'POSITIVO';
  end
else
  begin
    if numero < 0 then
      begin
        Label1.Caption := 'NEGATIVO';
      end
```

```
    else
      begin
        Label1.Caption := 'ZERO';
      end;
  end;
```

Carta e penna: ricopia questo codice sul quaderno e traccia una **graffa** da ogni begin al suo end (stesso numero). Poi disegna lo schema a frecce: due domande, tre risposte.

4. Riscaldamento (Form: Edit1, Button1, Label1)

Stessa Form per tutti e tre: una casella **Edit1** , un bottone **Button1** (Caption: **Controlla**), una scritta **Label1**. Doppio clic sul bottone e scrivi il codice tra begin ed end. Ricordati `var numero: Integer;` sopra il begin.

N.	Esercizio	Quante strade
R1	**Pari o dispari** : copia l'esempio del punto 1 e provalo con 4, 7, 0.	2 (un if)
R2	**Positivo, negativo o zero** : copia l'esempio del punto 3 e provalo con 5, -3, 0.	3 (if annidati)
R3	**Il voto** (fallo tu!): se il voto è minore di 6 scrivi **Insufficiente** ; altrimenti, se è minore di 8 scrivi **Sufficiente** ; altrimenti **Ottimo!**. Prova con 4, 6, 7, 9.	3 (if annidati)
R4	**Maggiorenne?** (fallo tu!): età minore di 14 -> **Bambino** ; minore di 18 -> **Ragazzo** ; altrimenti **Maggiorenne**.	3 (if annidati)

Aiuto per R3: è identico a R2: cambia solo le condizioni (`voto < 6`, `voto < 8`) e le scritte.

5. Il gioco: Indovina il numero

Indovina il numero

Ho pensato un numero da 1 a 100. Quale?

50 Prova Nuova partita

Troppo ALTO: prova un numero più piccolo

Ecco come sarà il tuo gioco (titolo e colori li scegli tu)

Livello 1 — Cosa deve fare

All'avvio il computer pensa un numero **segreto** da 1 a 100. Tu scrivi un numero e premi **Prova**. Il programma risponde: **Troppo BASSO** , **Troppo ALTO** oppure **BRAVO! Indovinato** (con il numero di tentativi). Il bottone **Nuova partita** pensa un altro numero.

Livello 2 — Un aiuto

Ti servono 2 variabili che **durano per tutta la partita** (segreto e tentativi): vanno in alto, sotto `Form1: TForm1;`. Il numero segreto si crea con `Random(100) + 1`, all'avvio (evento **OnCreate** della Form: doppio clic su un punto vuoto della Form). Nel bottone Prova usa un **if annidato** : prima chiedi "è uguale?", nell'else chiedi "è più piccolo?".

Livello 3 — I pezzi sulla Form

Pezzo	Name	Proprietà
Form	Form1	Caption = Indovina il numero
Label	Label1	Caption = Ho pensato un numero da 1 a 100. Quale?
Edit	EditNumero	Text = (vuoto)
Button	ButtonProva	Caption = Prova
Button	ButtonNuova	Caption = Nuova partita
Label	LabelRisposta	Caption = (vuoto) · Font: grande e colorato, a tuo gusto

Livello 4 — Il codice completo

A) In alto nel codice , sotto `Form1: TForm1;`, aggiungi le due variabili:

```
var
  Form1: TForm1;
  segreto: Integer;     // il numero pensato dal computer
  tentativi: Integer;   // quante prove hai fatto
```

B) Doppio clic su un punto vuoto della Form -> nasce **FormCreate** :

```
procedure TForm1.FormCreate(Sender: TObject);
begin
  Randomize;                    // mescola i numeri a caso
  segreto := Random(100) + 1;    // un numero da 1 a 100
```

```
  tentativi := 0;
end;
```

C) Doppio clic su Prova -> il cuore del gioco, con l'**if annidato** :

```
procedure TForm1.ButtonProvaClick(Sender: TObject);
var
  numero: Integer;
begin
  numero := StrToInt(EditNumero.Text);
  tentativi := tentativi + 1;
  if numero = segreto then
    begin
      LabelRisposta.Caption := 'Indovinato! Tentativi: ' + IntToStr(tentativi);
    end
  else
    begin
      if numero < segreto then
        begin
          LabelRisposta.Caption := 'Troppo BASSO: prova un numero più grande';
        end
      else
        begin
          LabelRisposta.Caption := 'Troppo ALTO: prova un numero più piccolo';
        end;
    end;
  EditNumero.Clear;
end;
```

D) Doppio clic su Nuova partita :

```
procedure TForm1.ButtonNuovaClick(Sender: TObject);
begin
  segreto := Random(100) + 1;
  tentativi := 0;
  LabelRisposta.Caption := 'Ho pensato un nuovo numero da 1 a 100';
end;
```

Se Lazarus dà errore su else: guarda l'end subito prima: non deve avere il **;**. **Se dà errore appena premi Prova con la casella vuota:** è normale, StrToInt vuole un numero (vedi sfida 3).

Fallo tuo: cambia il titolo della finestra, i colori, le frasi (anche spiritose) e il nome del gioco. Poi fallo provare a un compagno: chi indovina con meno tentativi?

6. Sfide per chi ha finito (sempre con gli if)

1. **Ci sei quasi!** Dentro “troppo alto” e “troppo basso” aggiungi un altro if: se la differenza è al massimo 5, scrivi **Ci sei quasi!** (aiuto: `if Abs(numero - segreto) <= 5 then`).
2. **Massimo 7 tentativi:** se tentativi arriva a 7 e non hai indovinato, scrivi **Hai perso! Il numero era** … (aiuto: `IntToStr(segreto)`).
3. **Fuori campo:** se il numero è minore di 1 o maggiore di 100 scrivi **Solo numeri da 1 a 100!** e non contare il tentativo (aiuto: `if (numero < 1) or (numero > 100) then`).
4. **Livelli:** aggiungi tre bottoni **Facile** (1-10), **Medio** (1-100), **Difficile** (1-1000).

Classe 2 · Dispensa if / then / else e if annidati · v1.0 · Non si consegna: si consegna il foglio del COMPITO.

v1.0

Compito — Indovina il numero

Classe 2 · Informatica · Lazarus · 05/10/2026 · si consegna su Classroom

Cosa consegni: il **Documento** già pronto nel compito su Classroom (c’è già il tuo nome), con **4 parti**.

Le 4 parti del Documento

1. **Gli screenshot del gioco che funziona** : tre immagini, una con **Troppo BASSO** , una con **Troppo ALTO** , una con **BRAVO! Indovinato**. (Premi Win + Shift + S, seleziona la finestra del gioco, poi nel Documento Ctrl + V.)
2. **Il codice del bottone Prova** : in Lazarus seleziona tutta la procedura **ButtonProvaClick** , da **procedure** fino al suo **end;** , copia e incolla. I rientri devono restare: if ed else allineati, begin ed end allineati.
3. **Spiegalo con parole tue** : a) perché qui serve un if **dentro** un altro if? b) cosa succede se scrivi il **;** prima di else?
4. **Riflessione** : cosa NON sono riuscito a fare e perché; cosa NON ho capito.

Come viene valutato (voto in decimi)

Cosa guardo	Punti
Il gioco funziona: i tre casi negli screenshot	**3**
If annidato corretto (uguale / più piccolo / più grande)	**2**
Rientri in ordine: if/else allineati, begin/end allineati, il resto indentato	**2**
Il gioco è tuo: titolo, frasi, colori	**1**
Spiegazione con parole tue	**1,5**
Riflessione	**0,5**
Totale (+1 bonus per ogni sfida della dispensa fatta bene, fino a 10)	**10**

Non hai finito? Consegna lo stesso quello che hai fatto e scrivilo nella riflessione: conta anche il pezzo fatto. Il bug è normale, succede a tutti i programmatori.

Carta e penna: sul quaderno disegna lo schema a frecce del bottone Prova (2 domande, 3 risposte) prima di scrivere il codice.

Classe 2 · Compito Indovina il numero · v1.0

L08 07/10/2026 — Quiz 1: cosa scrive il programma?

Libro parziale: lezioni/2026-10-07_quiz-if.md

Che cosa abbiamo fatto

1. **Quando:** 07/10/2026, Informatica.
2. **Argomenti:** Quiz di teoria sulle condizioni (if, then, else) e interrogazione.

La teoria e l’esercitazione

Questa lezione è stata di organizzazione e di avvio: non ha una dispensa scritta. Gli argomenti sono quelli elencati sopra.

L09 08/10/2026 — Il treno blindato: squadre e ruoli

Libro parziale: lezioni/2026-10-08_treno-blindato.md

Che cosa abbiamo fatto

1. **Quando:** 08/10/2026, Laboratorio.

2. Argomenti: Progetto del treno blindato: squadre con ruoli (CEO, Motore, Personaggi, Scenari, Test qualità), prime Pull Request entrate nel gioco.

La teoria e l’esercitazione

Giovedì 08/10/2026 · lo costruiamo tutti insieme, come in un’azienda di software

Il gioco: sei dentro un vagone. Il treno attraversa boschi, paesi, città, montagne e mare. I cattivi saltano fuori da dietro alberi e case: li tocchi prima che sparino. Il vetro antiproiettile regge 5 colpi.

La mia squadra: cosa fare adesso Gioca! Il progetto su GitHub

1. La nostra azienda (organigramma della lavagna)

Ruolo	Cosa cura	File
CEO	decide le priorità, approva i lavori con il prof, presenta il gioco	—
SCENARI	gli ambienti e il percorso: bosco, paese, città, montagna, mare	ambienti/*.json, percorso.json
PERSONAGGI	i cattivi, le facce della classe, il protagonista	cattivi/pcNN.json, facce.json, protagonista.json
MOTORE FISICA	velocità del treno, tempi dei cattivi, colpi del vetro, punti	regole.json
TEST QUALITÀ (la Triade)	provano il gioco su PC e telefono, trovano i difetti, controllano le richieste delle altre squadre	—
TUTTI	il proprio cattivo	cattivi/pcNN.json

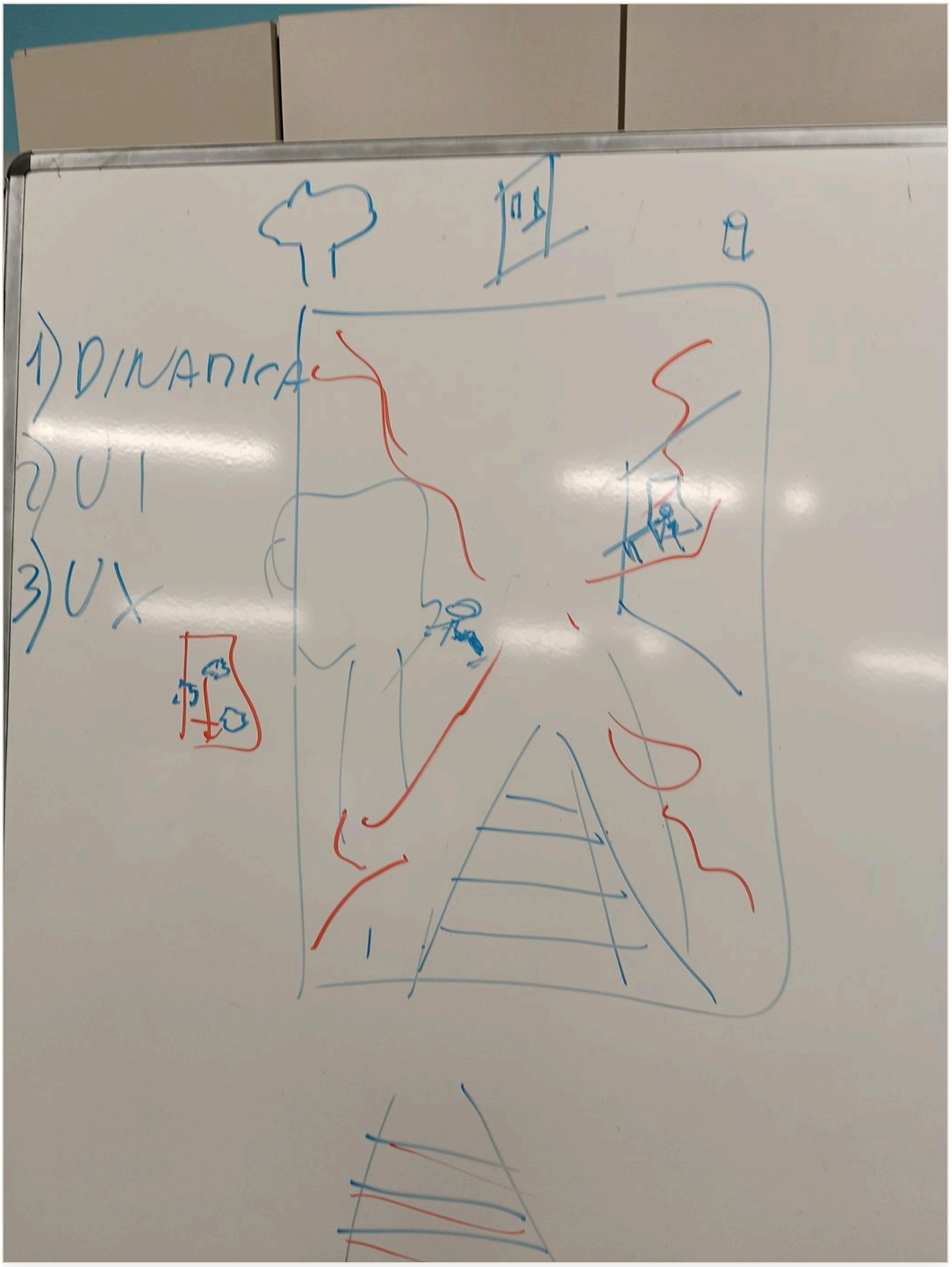
Il progetto (lavagna): si guarda avanti dalla cabina, lungo i binari; i cattivi spuntano dalle finestre delle case, da dietro gli alberi e dai barili. Tre cose da curare: **1) dinamica** (come si muove il gioco), **2) UI** (cosa si vede: vetro, punti, tachimetro), **3) UX** (come ci si sente a giocare).

Ruoli: il prof è il **product owner** (decide cosa entra e fa il merge); in ogni squadra c’è un **project manager**, a turno; tutti sono **sviluppatori**; un compagno di un’altra squadra fa la **revisione**.

Carta e penna: prima di toccare il computer, ogni squadra scrive su un foglio chi fa cosa (il **backlog**) e disegna il suo pezzo: il cattivo, il paesaggio, il percorso.

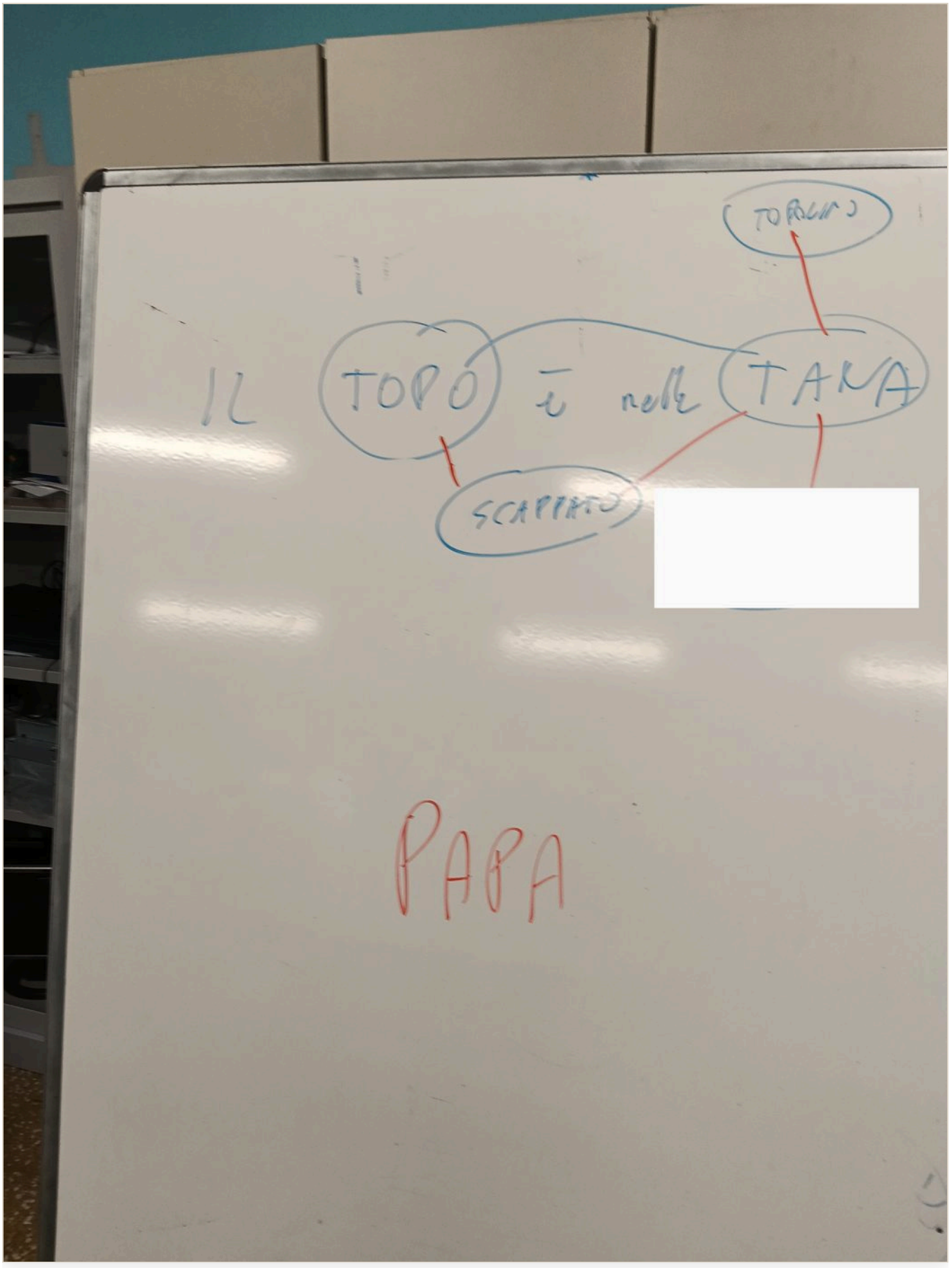
Le foto della lavagna: usatele per progettare

Il disegno del gioco: si guarda avanti dalla cabina, lungo i binari; i cattivi spuntano dalle finestre, dagli alberi e dai barili. 1) Dinamica 2) UI 3) UX.



Disegno del gioco alla lavagna

Come funziona Gemini (le parole collegate):



Mappa delle parole alla lavagna

La copertina con i personaggi (disegnata con l’IA):



Copertina del gioco

Carta e penna: ricopia sul quaderno il disegno del gioco e segna con un colore la parte della tua squadra.

Come funziona Gemini, il nostro assistente

Gemini ha letto moltissimi testi e ha imparato quali parole stanno insieme. Nella frase «Il **topo** è nella **tana**» collega «topo» a «scappato» e «tana» a «topolino»: quando gli scrivi, sceglie la parola che più probabilmente viene dopo. Per questo **più la tua frase è precisa, più la risposta è utile**. È un assistente: controlla sempre quello che ti risponde.

2. Il tuo cattivo

Regola: il cattivo lo inventi tu: niente nomi veri, niente parolacce. Le facce vere sono solo quelle dei compagni maggiorenni che hanno dato la loro foto (le inserisce la squadra Personaggi).

a) Inventalo con l'IA. Apri Gemini, incolla questa frase e cambia la parte tra parentesi quadre:

Frase per Gemini

```
Sto creando un videogioco per la scuola. Inventa un cattivo buffo che assalta un treno,
in stile cartone animato, non una persona reale. Il mio cattivo è [descrivi: colore,
carattere, cosa indossa]. Dammi: un nome di massimo 24 caratteri, un colore in codice
esadecimale (come #8a8f98) e una frase che grida di massimo 40 caratteri.
```

b) Scegli il tuo PC: — scegli —

Il nome del TUO file

Il tuo cattivo (poi cambia i valori con quelli di Gemini)

```
{
  "nome": "Il Bandito Nebbia",
  "colore": "#8a8f98",
  "occhi": 2,
  "cappello": 1,
  "bocca": 2,
  "velocita": 2,
  "frase": "Fermate quel treno!",
  "immagine": false
}
```

Campo	Cosa puoi mettere
occhi	1, 2 oppure 3
cappello	0 = niente, 1 = cappello, 2 = bandana sul viso
bocca	0 = denti, 1 = bocca aperta, 2 = zig-zag
velocita	1 = normale, 2 = veloce, 3 = velocissimo

3. Consegnalo su Classroom

Si consegna SOLO su Classroom, nel compito «Il treno blindato»: scrivi nel Documento e allega il disegno o la foto (bottone «Aggiungi o crea» ⟶ «File»). Non serve GitHub: gli sviluppatori (il prof con l'intelligenza artificiale) mettono la tua consegna su Git e la fanno entrare nella nuova versione del gioco. Così vedrai il tuo lavoro nel grafico delle versioni.

L10 09/10/2026 — Sicurezza 2: prevenzione e protezione

Libro parziale: lezioni/2026-10-09_sicurezza-2.md

Che cosa abbiamo fatto

1. **Quando:** 09/10/2026, 1 ora, Sicurezza.
2. **Argomenti:** Sicurezza sul lavoro (Regione), 2ª ora: prevenzione e protezione, i soggetti della sicurezza; quiz su Moduli.

La teoria e l'esercitazione

> ***Nota:*** *questo è il testo della teoria, da rivedere insieme al docente prima della lezione e poi da dare ai ragazzi. Linguaggio semplice, un passo alla volta. Nessun nome di allievo.*

1. Di cosa parliamo oggi

1. Nel Modulo 1 abbiamo visto le parole della sicurezza: pericolo, rischio, danno, prevenzione, protezione.
2. Oggi scopriamo **CHI** si occupa della sicurezza in un'azienda: non è una persona sola, è una squadra.

3. Immagina una vera azienda di informatica (un'officina che ripara computer e monta reti). Dentro ci sono tante persone, e ognuna ha un compito preciso anche per la sicurezza.

4. Alla fine parliamo della cosa più importante per te: i tuoi **diritti** e i tuoi **doveri** quando lavori (anche quando vai in stage).

2. I protagonisti della sicurezza (la squadra)

2.1 Il datore di lavoro

1. È chi **comanda** l'azienda e decide le cose importanti (compra le macchine, assume le persone, spende i soldi).

2. Per la sicurezza è il **primo responsabile**: se qualcosa va storto, risponde lui.

3. Compiti suoi: valutare i rischi, scrivere il documento dei rischi, dare i mezzi di protezione, organizzare la formazione.

4. Esempio in officina: è il titolare che ha aperto l'azienda di informatica. Decide lui di comprare i banchi da lavoro giusti e le prese elettriche a norma.

2.2 Il dirigente e il preposto

1. Il **dirigente** è un capo che dà ordini e organizza il lavoro al posto del datore di lavoro (come un vice).

2. Il **preposto** è chi **controlla sul campo**, mentre il lavoro si fa: sta lì, guarda, e se vede qualcuno che sbaglia lo corregge subito.

3. Parola facile: il preposto è il **capo-reparto**, quello che ti sta vicino e ti dice "metti i guanti" o "non togliere quella protezione".

4. Esempio in officina: il capo del reparto riparazioni che controlla che tutti stacchino la corrente prima di aprire un computer.

2.3 Il RSPP (Responsabile del Servizio di Prevenzione e Protezione)

1. È l'**esperto** della sicurezza: studia i rischi dell'azienda e dice come difendersi.

2. Sigla: **RSPP** = Responsabile del Servizio di Prevenzione e Protezione.

3. Non comanda come il datore di lavoro: è un **consulente**, uno che sa e consiglia.

4. Esempio in officina: controlla che i cavi elettrici non siano scoperti e propone dove mettere l'estintore.

2.4 Il medico competente

1. È il **medico** che fa le **visite mediche** ai lavoratori per controllare che il lavoro non faccia male alla salute.

2. Parola facile: “competente” vuol dire che è esperto proprio di salute sul lavoro.

3. Esempio in officina: visita chi sta tante ore al computer e controlla la vista e la schiena.

2.5 Il RLS (Rappresentante dei Lavoratori per la Sicurezza)

1. È un lavoratore **scelto (eletto) dai colleghi** per **parlare al posto loro** con il datore di lavoro sui temi della sicurezza.

2. Sigla: **RLS** = Rappresentante dei Lavoratori per la Sicurezza.

3. Parola facile: è la **voce dei lavoratori**. Se un collega ha paura di lamentarsi, lo fa lui per tutti.

4. Esempio in officina: porta al titolare la richiesta dei colleghi di avere sedie migliori per non farsi male alla schiena.

2.6 Il lavoratore (anche tu, in stage)

1. È chi **fa il lavoro**. Anche lo **studente in stage** e l’**apprendista** sono lavoratori per la legge.

2. Non è solo un numero: ha il dovere di **collaborare** alla sicurezza di tutti.

3. Cosa deve fare: rispettare le regole, usare i mezzi di protezione (i DPI), segnalare i pericoli che vede.

4. Esempio in officina: se vedi un cavo per terra che fa inciampare, lo dici subito al preposto, non aspetti che qualcuno cada.

3. Tabella “chi fa cosa”

Soggetto	Chi è, con parole semplici	Cosa fa per la sicurezza
Datore di lavoro	Chi comanda l’azienda	È il primo responsabile: valuta i rischi, dà i mezzi di protezione, organizza la formazione
Dirigente	Un capo che organizza al posto del datore	Dà ordini e fa rispettare le regole di sicurezza
Preposto	Il capo-reparto che ti sta vicino	Controlla sul campo e corregge subito chi sbaglia
RSPP	L’esperto di sicurezza	Studia i rischi e consiglia come difendersi
Medico competente	Il medico del lavoro	Fa le visite mediche ai lavoratori
RLS	La voce dei lavoratori, eletto dai colleghi	Parla con l’azienda per conto dei lavoratori
Lavoratore (anche studente/apprendista)	Chi fa il lavoro	Rispetta le regole, usa i DPI, segnala i pericoli

4. I tuoi diritti e i tuoi doveri (la parte più importante)

4.1 I tuoi DIRITTI (cosa ti DEVONO dare)

1. **Formazione**: ti devono spiegare come lavorare senza farti male (come questo corso).
2. **Informazione**: ti devono dire quali sono i pericoli del posto dove lavori.
3. **DPI**: ti devono dare i **Dispositivi di Protezione Individuale** (guanti, occhiali, scarpe antinfortunistiche…) senza farteli pagare.
4. **Visita medica**: hai diritto ai controlli del medico competente, quando servono.
5. Sigla: **DPI** = Dispositivi di Protezione Individuale (le cose che indossi per proteggerti).

4.2 I tuoi DOVERI (cosa DEVI fare tu)

1. **Rispettare le regole** e le istruzioni che ti danno.
2. **Usare bene i DPI** e le macchine, come ti hanno insegnato.

3. **Non togliere e non rompere le protezioni** (per esempio il carter di una macchina o la messa a terra di una presa).

4. **Segnalare subito** i pericoli e i guasti che vedi, al preposto o al capo.

5. Regola d'oro: la sicurezza non è "solo degli altri". Anche tu fai parte della squadra.

> ***Attenzione:*** *togliere una protezione per fare prima è l'errore più grave. Sembra che faccia risparmiare tempo, ma è proprio lì che succedono gli infortuni più brutti.*

5. Chi controlla da fuori (gli organi di vigilanza) — brevissimo

1. **ASL / ATS**: i medici e i tecnici del territorio che controllano che le aziende rispettino le regole di sicurezza.

2. **INAIL**: l'ente che **assicura** i lavoratori: se ti fai male sul lavoro, l'INAIL ti aiuta economicamente.

3. **Ispettorato del lavoro**: i controllori che vengono in azienda a verificare che tutto sia in regola.

4. Sigle: **ASL** = Azienda Sanitaria Locale · **ATS** = Agenzia di Tutela della Salute · **INAIL** = Istituto Nazionale Assicurazione Infortuni sul Lavoro.

6. Saperlo spiegare (prova del nove)

> ***Nota:*** *se sai rispondere a voce, con parole tue, a queste domande, vuol dire che hai capito davvero. Provale con un compagno.*

1. In un'azienda di informatica, chi è il **primo responsabile** della sicurezza e perché?

2. Qual è la differenza tra il **preposto** (il capo-reparto) e il **RSPP** (l'esperto)?

3. Chi è il **RLS** e perché è importante che sia **scelto dai lavoratori**?

4. Di' **un tuo diritto** e **un tuo dovere** da lavoratore, con un esempio concreto.

L11 12/10/2026 — Versioning e Agile

Libro parziale: lezioni/2026-10-12_versioning-agile.md

Che cosa abbiamo fatto

1. **Quando:** 12/10/2026, 3 ore, Laboratorio.
2. **Argomenti:** Versioning e teoria Agile (Manifesto e 10 regole); Pull Request sul gioco della classe; backlog con pesi di Fibonacci; 20 versioni del gioco (v0.12-v0.31) con changelog e grafo.

La teoria e l'esercitazione

Testo per la tua AI: chiedile di spiegarti ogni parola nella tua lingua.

01 I link della lezione

1. Il grafo delle versioni (clicca i pallini)

```
https://nicolaregge-pulse.github.io/corso-informatica/giochi/grafo/
```

1. Tutte le versioni del vostro treno, giocabili

```
https://nicolaregge-pulse.github.io/corso-informatica/giochi/treno-blindato/versioni/
```

1. Il changelog del treno (la lista dei cambiamenti)

```
https://github.com/nicolaregge-pulse/corso-informatica/blob/main/docs/giochi/treno-
blindato/CHANGELOG.md
```

1. La storia dei commit del ramo treno-2inf

```
https://github.com/nicolaregge-pulse/corso-informatica/commits/treno-2inf
```

1. Il grafo vero di Git su GitHub (Network)

```
https://github.com/nicolaregge-pulse/corso-informatica/network
```

1. Il fork della 3INF: il loro treno

```
https://nicolaregge-pulse.github.io/corso-informatica/giochi/treno-3inf/
```

1. La versione 2.0 su Godot

```
https://nicolaregge-pulse.github.io/corso-informatica/giochi/treno-godot/
```

02 Le 10 parole, con il nostro gioco

1. Versione. Una «fotografia» del progetto in un momento preciso, con un numero. Il vostro treno è passato da v0.0 (prima idea, finestrino laterale) a v0.11 (i punteggi si salvano sempre): 12 versioni in un giorno.

1.2 Prova: Apri «Tutte le versioni» e gioca la v0.0 e la v0.11: cosa è cambiato?

1. Git e GitHub. Git è il programma che ricorda TUTTA la storia del progetto: chi ha cambiato cosa, quando e perché. GitHub è il sito dove il progetto (il repository) sta online. Il nostro repository si chiama corso-informatica.

2.2 Prova: Apri «La storia dei commit»: ogni riga è un salvataggio.

1. Commit. Un salvataggio nella storia di Git, con un messaggio che spiega il cambiamento. Esempio vero di ieri: 7724900
«Treno blindato v0.11: i punteggi si salvano sempre». Il numero strano (7724900) è il codice del commit.

3.2 Prova: Trova nella storia il commit della v0.5 (la Pull Request #104).

1. Branch (ramo). Una linea di lavoro separata. Si apre un ramo per provare una cosa nuova senza rompere il gioco che funziona. Il vostro gioco sta nel ramo treno-2inf; il prof ha provato le «immagini della classe» su un ramo a parte.

4.2 Prova: Nel grafo: quante linee (rami) vedi?

1. Fork (copia). La copia di un progetto intero che cresce per conto suo. Ieri la 3INF ha fatto il fork del vostro treno alla v0.11 e lo ha portato alla v1.4 (montagna, Black Panter, Nash…); anche la 1INF.

5.2 Prova: Nel grafo: da quale vostra versione parte la linea verde della 3INF?

1. Push. Mandare i propri commit dal PC al repository online (GitHub). Ieri, dopo ogni versione, il prof ha fatto il push: per questo il gioco sul sito cambiava da solo.

6.2 Prova: Perché il gioco sul telefono cambiava senza scaricare niente?

1. Pull. Il contrario del push: prendere da GitHub l’ultima versione e portarla sul proprio PC. La v0.11 del treno «chiede sempre alla rete la versione nuova»: è un pull automatico.

7.2 Prova: Quando fai un pull, cosa ricevi?

1. Pull Request (proposta di modifica).
«Ho fatto una modifica: la vuoi nel progetto?». Qualcuno la controlla e, se va bene, fa il MERGE (la unisce). Ieri le vostre richieste su Classroom erano Pull Request: la #104 della squadra MOTORE FISICA è diventata la v0.5.

8.2 Prova: Chi decide se una Pull Request entra nel gioco?

1. Changelog (registro dei cambiamenti). La lista, versione per versione, di cosa è cambiato e perché. Il treno ha il suo CHANGELOG.md: chi gioca sa cosa c'è di nuovo, chi programma sa dove cercare un errore.

9.2 Prova: Apri il changelog: qual è la novità della v0.7?

1. Numeri delle versioni (major.minor). v0.x = prove; 1.0 = prima versione stabile (il fork della 3INF parte da 1.0); 1.1, 1.2 = piccole aggiunte (minor); 2.0 = cambio grosso (major): il gioco rifatto con Godot è la v2.0.

10.2 Prova: Perché la versione Godot non si chiama v1.5?

L12 13/10/2026 — Verifica sul versioning

Libro parziale: lezioni/2026-10-13_verifica-versioning.md

Che cosa abbiamo fatto

1. Quando: 13/10/2026, Laboratorio.

2. Argomenti: Ripasso e verifica sul versioning; revisione delle versioni del treno.

La teoria e l'esercitazione

Le schede della lezione

1. Le versioni

> Una **versione** è un salvataggio del programma con un **numero** e una riga che dice **cosa cambia**. Se cambia il numero dopo il punto (v0.30 ⟶ v0.31) è un passo piccolo, **minor**. Se cambia il primo numero (v1.4 ⟶ v2.0) è un cambio grande, **major**: per esempio un motore nuovo, come Godot. Il **changelog** è il registro di tutte le modifiche, versione per versione.

2. Git e GitHub

Git è il programma che tiene tutte le versioni di un progetto. Ogni salvataggio si chiama **commit**. Il **repository** è la cartella del progetto con tutta la sua storia. **GitHub** è il sito che tiene i repository su Internet e li mostra a tutti: è lì che sta il nostro treno. Un **ramo** (branch) è una strada parallela di lavoro; un **fork** è una copia del progetto (la 3INF ha fatto il fork del nostro treno); il **merge** unisce un ramo dentro un altro.

3. La Pull Request e il backlog

Una **Pull Request** (PR) è una **richiesta di modifica scritta**: Titolo, Cosa cambia, Dove, Perché. Tutte le richieste vanno nel **backlog**, la coda dei lavori. Ognuna vale uno, chiunque la faccia; il **peso** dice quanto lavoro serve sul codice (1 colore, 2 numero, 3 personaggio, 5 ambiente, 8 regola nuova). Si fanno prima le più leggere; a parità di peso, chi è arrivato prima. È il metodo **Agile**: piccoli passi, sbagliare presto e correggere presto.

4. Il nostro sistema, pezzo per pezzo

1) Su **Google Classroom** c'è un solo compito con un link fisso. 2) La PR la mandi dal **Modulo «Consegna»** (Google Moduli). 3) Uno **script** (un piccolo programma di Google, Apps Script) ogni minuto porta le consegne nel **repository** del corso su **GitHub**. 4) **Claude**, l'intelligenza artificiale che lavora con il prof, legge la PR, le dà un peso nel backlog, **scrive il codice** della modifica, la prova nel browser e pubblica la versione nuova. 5) **GitHub Pages** mette il gioco e le pagine (backlog, voti, monitor, novità) sul sito, e si aggiornano da sole. 6) Il prof decide: sì o no alla PR, e il voto.

5. Chi fa cosa

Tu hai l'idea e la scrivi bene. **Il prof** decide cosa entra e dà il voto. **Claude** scrive il codice e lo prova. **GitHub** tiene tutte le versioni, così se una modifica rompe il gioco si torna indietro. Da oggi il treno passa su **Godot**: è un cambio major, e ogni classe avrà il suo **ramo** che parte dalla base del prof.

Versione	Cosa è entrato	Cosa funziona	Cosa si può migliorare
v0.12	La versione serale	Con una sola idea il gioco cambia atmosfera: il tramonto lo rende più bello da guardare.	Al tramonto i cattivi scuri si staccano meno dallo sfondo: vanno provati con colori più chiari.
v0.13	Nuovo livello, nuovo scenario	Ogni livello è un posto nuovo: chi gioca sente che sta andando avanti.	Il cambio di scenario arriva di colpo: una scritta o un attimo di pausa lo annuncerebbe meglio.
v0.14	Ultimo colpo	L'ultimo colpo dà ancora una possibilità e crea tensione.	Con il vetro tutto crepato l'ultimo colpo si gioca quasi alla cieca: per questo nella base del prof le crepe partono dai bordi.
v0.15	La difficoltà cresce a ogni livello	Il gioco resta interessante anche per chi è bravo.	Nessuno ha ancora misurato quanto diventa difficile dopo il livello 5: serve un collaudo.
v0.16	Vetro da 10 colpi	Partite più lunghe e meno frustrazione per chi comincia.	Con 10 colpi i primi livelli diventano facili: la difficoltà va bilanciata insieme al vetro.
v0.17	I cattivi non nascono più troppo vicino	Difetto vero, trovato giocando: è il lavoro di chi fa il collaudo.	Ora i cattivi sono più lontani: sul telefono vanno controllati, potrebbero essere piccoli da toccare.
v0.18	BANG! e il cattivo cade a terra	Il colpo si sente e si vede: soddisfazione immediata.	Tre richieste sulla stessa cosa: prima di mandarla, conviene guardare il backlog e unirsi a quella che c'è già.
v0.19	Il mirino rosso	Il mirino rosso rende la mira più precisa e il gioco più «da gioco».	Sul telefono il mirino non c'è, perché si tocca con il dito: l'idea vale solo per il PC.
v0.20	I cattivi si vedono sempre	Corregge un difetto che rovinava la partita.	Il difetto era nato da una versione precedente: ogni modifica va provata giocando prima di pubblicarla.
v0.21	Le luci dei palazzi non lampeggiano più	Tolto un lampeggio fastidioso, che può anche dare fastidio agli occhi.	Con le luci ferme il paese sembra meno vivo: si potrebbero accendere piano piano.

Versione	Cosa è entrato	Cosa funziona	Cosa si può migliorare
v0.22	Il record accanto ai punti	La sfida continua contro se stessi: il record è sempre lì accanto ai punti.	Il record resta solo su quel computer: su un altro PC si riparte da zero.
v0.23	Il bottone Pausa	Indispensabile in classe: si può fermare la partita (anche con il tasto P o Esc) e riprenderla.	Il bottone è piccolo: sul telefono va provato che si tocchi bene.
v0.24	Più avanti, più punti	Premia chi resiste di più: andare avanti conviene.	I punti crescono in fretta: i record dei primi giorni diventano impossibili da battere?
v0.25	Colpo preciso	Premia la precisione, non solo la velocità.	Non si capisce subito quando un colpo è «preciso»: serve una scritta che lo dica.
v0.26	Pioggia e nebbia	Quattro richieste sulla stessa idea diventano una versione sola: lavoro di squadra vero.	Con pioggia e nebbia si vede poco: va bene per i livelli alti, ma all'inizio potrebbe scoraggiare.
v0.27	Binari gialli	I binari gialli si vedono meglio: modifica piccola, effetto grande.	Il giallo non c'entra molto con il bosco bruciato: un colore più scuro potrebbe stare meglio.
v0.28	La barra del prossimo livello	Si vede quanto manca al prossimo livello: dà un obiettivo vicino.	La barra è in alto al centro, vicino ad altre scritte: va controllato che non copra i cattivi.
v0.29	Le statistiche a fine partita	A fine partita si vede cosa si è fatto bene: precisione e colpi sbagliati.	Le statistiche spariscono con la partita dopo: si potrebbero tenere le migliori.
v0.30	La sfera d'oro: punti x2	La sfera d'oro dà un momento di sorpresa e cambia il ritmo.	Quando si colpisce la sfera, non si vede bene quanto resta dei 10 secondi: serve un contatore.
v0.31	Gli uccelli nel cielo	Gli uccelli rendono il cielo vivo: un dettaglio che fa sembrare il mondo vero.	Gli uccelli possono distrarre: non devono sembrare cattivi da colpire.

Domande per l'analisi in classe

1. Quale versione ha migliorato di più il gioco? Perché?
2. Quale miglioramento proposto qui sopra faresti per primo? Scrivilo come Pull Request: Titolo, Cosa cambia, Dove, Perché.
3. Trovi un difetto che qui non c'è? Provalo giocando e scrivilo come Pull Request.

L13 15/10/2026 — Verifica rapida e il ciclo FOR

Libro parziale: lezioni/2026-10-15_ciclo-for-1.md

Che cosa abbiamo fatto

1. **Quando:** 15/10/2026, Informatica.
2. **Argomenti:** Verifica rapida di versioning su Moduli; il ciclo FOR in Lazarus, prima parte.

La teoria e l'esercitazione

Le schede della lezione

1. Che cos'è un ciclo

> Un **ciclo** fa ripetere al computer la stessa cosa tante volte, senza riscriverla. Per scrivere CIAO 10 volte, senza ciclo servono 10 righe uguali; con il ciclo **for** ne bastano 3. In Lazarus finora il programma **reagiva** a un clic e faceva una cosa sola; con il for, dentro lo stesso clic, il programma **ripete**.

2. La lavagna: FOR X := 1 TO 10 DO

```
for x := 1 to 10 do begin   writeln('CIAO'); end;
```

Si legge così: **per** x che va **da 1 a 10, fai** quello che c'è tra **begin** ed **end**.
x è il **contatore**: una variabile **intera** (`integer`) che a ogni giro aumenta di 1. Va dichiarata prima, come tutte le variabili: `var x: integer;`
:= dà il valore di partenza (come sempre in Pascal). **to** dice dove arriva. **do** vuol dire «fai».
writeln scrive e va a capo (**ln** = line, riga); **write** invece scrive e resta sulla stessa riga.

3. Il contatore cambia a ogni giro

```
for giro := 7 to 12 do begin   writeln(IntToStr(giro)); end;
```

Il nome del contatore lo scegli tu: alla lavagna il prof ha usato il nome di un compagno. Qui si chiama **giro**.
Il ciclo parte da 7 e arriva a 12: scrive **7, 8, 9, 10, 11, 12**, uno per riga. I giri sono **6**: si contano così, **fine − inizio + 1** (12 − 7 + 1 = 6).
IntToStr trasforma un numero intero in testo (Int = intero, Str = stringa, testo), come facevate nella calcolatrice per mettere un numero nella Caption.

4. In Lazarus, con il form: si scrive nel Memo

Alla lavagna c'è **writeln**, che scrive in una finestra di solo testo. In un programma con il **form** (bottoni e caselle) writeln non si vede: si usa un **Memo** (TMemo, una casella di testo con tante righe). Ogni giro aggiunge una riga:

```
for giro := 1 to 10 do begin   Memo1.Lines.Add('CIAO'); end;
```

Memo1.Lines.Add = «al Memo1, alle sue righe, aggiungi». Con il numero del giro:

```
Memo1.Lines.Add(IntToStr(giro) + ' - CIAO');
```

Il **+** tra due testi li attacca uno all'altro.

5. Gli errori di tutti (anche dei programmatori veri)

1) Il contatore **non dichiarato**: manca `var giro: integer;` sopra il begin del bottone. Lazarus dice «Identifier not found».
2) Il **punto e virgola dopo do**: `for giro := 1 to 10 do;` ripete 10 volte... niente, e la riga sotto la fa una volta sola.
3) **begin** senza **end;**: Lazarus non compila.
4) Il numero nel Memo **senza IntToStr**: errore, perché il Memo vuole testo.
5) I **rientri** (regola del prof): **begin** ed **end** sullo stesso rientro, e tutto quello che sta dentro si sposta a destra di **2 spazi**. Così si vede subito che cosa si

ripete. In Lazarus: selezioni le righe e premi **Ctrl+Shift+I** per spostarle a destra (**Ctrl+Shift+U** per tornare a sinistra). Codice tutto attaccato a sinistra = codice che funziona ma che nessuno riesce a leggere: in un'azienda non passa.

6. Il programma della lavagna: da … a …

L'**interfaccia** (UI, quello che si vede) disegnata alla lavagna: in alto **da** [5] **a** [12], al centro un Memo grande, in basso un bottone. Premi il bottone e nel Memo compaiono i numeri da 5 a 12, uno per riga.

I componenti: **TLabel** per le scritte «da» e «a», **TEdit** per le due caselle dei numeri, **TMemo** per l'elenco, **TButton** per partire. Nel Memo la proprietà **ScrollBars** va messa a **ssBoth**: compaiono le **barre di scorrimento**, e quando i numeri sono tanti (da 1 a 100) con la barra scorri giù e vedi anche i valori sotto.

Il codice del bottone:

```
var giro, inizio, fine: integer; begin   inizio := StrToInt(Edit1
```

StrToInt fa il contrario di IntToStr: trasforma il testo della casella in un numero. **Memo1.Clear** pulisce il Memo prima di ricominciare.

7. LA PRATICA: il tuo «da … a …» in Lazarus

Un passo alla volta, in Lazarus. Ogni passo è una cosa sola.

1) Apri Lazarus e scegli **Progetto ⟶ Nuovo progetto ⟶ Applicazione**, poi **Ok**.

2) Dalla scheda **Standard** metti sul form un **TLabel**; nell'Ispettore la sua **Caption** diventa **da**.

3) Metti accanto un **TEdit** (Edit1) e cancella il testo della proprietà **Text**.

4) Metti un altro **TLabel** con Caption **a**.

5) Metti accanto un altro **TEdit** (Edit2), con Text vuoto.

6) Metti al centro un **TMemo** grande; nell'Ispettore metti **ScrollBars** a **ssBoth**.

7) Metti in basso un **TButton** con Caption **Conta!**

8) Fai **doppio clic** sul bottone: si apre `Button1Click`.

9) Scrivi il codice della scheda 6, riga per riga, **con i rientri di 2 spazi** come nella scheda (begin ed end allineati, quello che sta dentro più a destra).

10) Premi **F9**, scrivi 5 e 12 nelle caselle, premi **Conta!**: compaiono i numeri da

5 a 12. **FATTO!**
10b) Prova da 1 a 100: i numeri non ci stanno, usa la **barra di scorrimento** a destra del Memo per vedere quelli sotto.
11) **Fallo tuo**: accanto al numero metti una frase tua, per esempio `IntToStr(giro) + ' - forza!'`, e cambia il titolo del form (Caption del form) con un nome tuo.
12) **Sfida**: una terza casella con un numero e nel Memo la sua tabellina: `IntToStr(giro) + ' x 7 = ' + IntToStr(giro * 7)`
Ti blocchi? Scrivi la domanda all'assistente, qui sotto: ti risponde.

01 Che cosa costruisci oggi

Un’app con due caselle: **da** [5] **a** [12]. Premi il bottone e nel riquadro grande compaiono i numeri da 5 a 12, uno per riga. Il computer li scrive da solo, con il **ciclo FOR**: una riga di codice che si ripete tante volte.

> ***Nota:*** *ti blocchi? Apri la pagina del percorso e scrivi la domanda nella casella «Chiedi aiuto all’assistente del Prof Regge». L’indirizzo è nel compito di Classroom.*

02 La teoria in 5 righe

1. Il **ciclo FOR** ripete quello che c’è tra **begin** ed **end**.
2. `for giro := 5 to 12 do` vuol dire: per giro che va da 5 a 12, fai.
3. **giro** è il **contatore**: una variabile intera (`integer`) che a ogni giro aumenta di 1. Va dichiarata con `var`.
4. I giri sono: fine meno inizio più 1. Da 5 a 12 sono 8 giri.
5. **StrToInt** trasforma il testo di una casella in numero; **IntToStr** trasforma un numero in testo, per scriverlo nel riquadro.

03 I componenti della lavagna

Componente	A cosa serve	Che cosa cambi nell'Ispettore
TLabel	la scritta «da»	Caption: da
TEdit (Edit1)	la casella del primo numero	Text: vuoto
TLabel	la scritta «a»	Caption: a
TEdit (Edit2)	la casella del secondo numero	Text: vuoto
TMemo (Memo1)	il riquadro grande dove compaiono i numeri	ScrollBars: ssBoth (le barre per scorrere)
TButton (Button1)	il bottone che fa partire il ciclo	Caption: Conta!

04 I passi in Lazarus

Un passo alla volta. Ogni passo è una cosa sola.

1. Apri Lazarus.

2. In alto clicca il menu **Progetto**, poi **Nuovo progetto**, poi **Applicazione**, poi **Ok**.

3. In alto, nella scheda **Standard** dei componenti, clicca **TLabel**, poi clicca sul form in alto a sinistra.

4. Nell'**Ispettore oggetti** (a sinistra), alla riga **Caption**, scrivi «da» e premi Invio.

5. Nella scheda Standard clicca **TEdit**, poi clicca sul form accanto a «da».

6. Nell'Ispettore, alla riga **Text**, cancella il testo e premi Invio.

7. Metti un altro **TLabel** più a destra; nella Caption scrivi «a».

8. Metti un altro **TEdit** accanto ad «a»; cancella il suo Text.

9. Nella scheda Standard clicca **TMemo** e disegna sul form un riquadro grande, al centro.

10. Nell'Ispettore, alla riga **ScrollBars**, scegli **ssBoth**: così compaiono le barre per scorrere e vedere i valori sotto.

11. Metti un **TButton** in basso; nella Caption scrivi «Conta!».

12. Fai **doppio clic** sul bottone: si apre il codice, dentro `Button1Click`.

13. Scrivi il codice qui sotto, riga per riga, con i **rientri**: begin ed end sullo stesso rientro, quello che sta dentro 2 spazi più a destra. Per spostare più righe: selezionale e premi Ctrl+Shift+I.

```
procedure TForm1.Button1Click(Sender: TObject);
var giro, inizio, fine: integer;
begin
  inizio := StrToInt(Edit1.Text);
  fine := StrToInt(Edit2.Text);
  Memo1.Clear;
  for giro := inizio to fine do
  begin
    Memo1.Lines.Add(IntToStr(giro));
  end;
end;
```

1. Premi **F9**: l'app parte.
2. Scrivi 5 nella prima casella e 12 nella seconda, poi premi **Conta!**. Compaiono i numeri da 5 a 12. **FATTO!**
3. Prova da 1 a 100: i numeri non ci stanno tutti. Usa la **barra di scorrimento** a destra del riquadro per vedere quelli sotto.

05 Fallo tuo

1. Accanto al numero metti una frase tua: `Memo1.Lines.Add(IntToStr(giro) + ' - forza!');`
2. Cambia il titolo della finestra: clicca sul form vuoto e nell'Ispettore cambia la sua **Caption** con un nome tuo.
3. **Sfida:** la tabellina del 7: `Memo1.Lines.Add(IntToStr(giro) + ' x 7 = ' + IntToStr(giro * 7));`

06 Gli errori di tutti

Che cosa vedi	Perché	Come si sistema
«Identifier not found» su giro	il contatore non è dichiarato	scrivi `var giro, inizio, fine: integer;` sopra il begin
il ciclo scrive una riga sola	c'è un punto e virgola dopo `do`	togli il `;` dopo do
errore sulla riga del Memo	manca IntToStr	scrivi `IntToStr(giro)`
errore quando premi Conta!	una casella è vuota o non ha un numero	scrivi un numero in tutte e due le caselle
Lazarus non compila	manca un `end;`	ogni begin ha il suo end
il codice è tutto attaccato a sinistra	mancano i rientri	seleziona le righe di dentro e premi Ctrl+Shift+I: 2 spazi a destra

> ***Nota:*** *sbagliare è normale, succede a tutti i programmatori, anche ai professionisti. Leggi il messaggio di Lazarus, guarda la tabella, prova di nuovo.*

L14 16/10/2026 — Sicurezza 3: organizzazione della prevenzione

Libro parziale: lezioni/2026-10-16_sicurezza-3.md

Che cosa abbiamo fatto

1. **Quando:** 16/10/2026, 1 ora, Sicurezza.
2. **Argomenti:** Sicurezza sul lavoro (Regione), 3ª ora: organizzazione della prevenzione aziendale (video e slide ufficiali).

La teoria e l'esercitazione

Le schede della lezione

1. Chi si occupa della sicurezza

> Il **datore di lavoro** è il primo responsabile. Il **dirigente** organizza; il **preposto** (capo-reparto) controlla sul campo. Il **RSPP** è l'esperto che studia i rischi. Il **medico competente** fa le visite. Il **RLS** è eletto dai colleghi ed è la loro voce.

2. I tuoi diritti e doveri

> **Diritti** del lavoratore (anche in stage): formazione, informazione, **DPI gratis**, visita medica. **Doveri**: rispettare le regole, non togliere le protezioni, usare i DPI, **segnalare i pericoli**.

3. Chi controlla da fuori

> Gli **organi di vigilanza** (ASL/ATS, Ispettorato del lavoro) controllano dall'esterno. L'**INAIL** assicura: se ti fai male sul lavoro, ti aiuta economicamente.

La pagina della lezione, con il ripasso: https://nicolaregge-pulse.github.io/corso-informatica/verifica/?b=2inf-sicurezza-3ora

01 La squadra della prevenzione

La teoria della **3ª ora** è in questa pagina: leggi le 4 schede, una alla volta.

In azienda la sicurezza è organizzata come una squadra. Il **datore di lavoro** decide e risponde. Il **Servizio di Prevenzione e Protezione** (SPP), guidato dal **RSPP**, studia i rischi e propone le soluzioni. Il **medico competente** controlla la salute. Il **RLS** parla a nome dei lavoratori. I **preposti** controllano sul campo.

02 Il DVR: la mappa dei rischi

Il **DVR** (Documento di Valutazione dei Rischi) è il documento dove l'azienda scrive **tutti i rischi** di ogni lavoro e **che cosa fa** per ridurli. Lo firma il datore di lavoro, con l'RSPP e il medico competente, sentito l'RLS. Il datore di lavoro **non può delegarlo** a nessuno. Si aggiorna quando cambia qualcosa: una macchina nuova, un reparto nuovo, dopo un infortunio. Almeno **una volta l'anno** c'è la **riunione periodica** della squadra della sicurezza.

03 Le emergenze: chi interviene

Ogni azienda ha gli **addetti alle emergenze**: gli **addetti antincendio** (usano gli estintori e guidano l'evacuazione) e gli **addetti al primo soccorso** (aiutano chi si fa male e chiamano il **112**). C'è un **piano di emergenza**: vie di fuga, **uscite di emergenza** con il cartello verde, **punto di raccolta** fuori dall'edificio. Durante

l’evacuazione: niente ascensore, niente corsa, non si torna indietro a prendere le cose.

04 Formazione, informazione, addestramento

Informazione: ti dicono quali rischi ci sono e chi sono le persone della sicurezza. **Formazione**: un corso per capire (come questo: 4 ore generali + le ore specifiche). **Addestramento**: provi davvero, con le mani, l’attrezzo o il DPI, con qualcuno che ti guarda. Si fanno **durante l’orario di lavoro** e **non costano niente al lavoratore**. Valgono anche per lo **studente in stage**.

05 Che cosa fare oggi

1. La teoria della 3ª ora è in questa pagina: leggi le 4 schede, una alla volta.
2. Fai il ripasso sulla pagina della lezione.
3. Fai il quiz su Google Moduli (link su Classroom) e premi Invia.

L15 19/10/2026 — Il ciclo FOR: il programma Contatore

Libro parziale: lezioni/2026-10-19_ciclo-for-2.md

Che cosa abbiamo fatto

1. **Quando:** 19/10/2026, 3 ore, Laboratorio.
2. **Argomenti:** Ciclo FOR in Lazarus, seconda parte: programma «Contatore» con due TEdit (da… a…), TButton e TMemo; sleep(1000) per vedere il ciclo che conta.

La teoria e l’esercitazione

Le schede della lezione

1. Giovedì: il ciclo for in una riga

> Giovedì abbiamo visto il **ciclo for**: fa ripetere al computer la stessa cosa tante volte. Alla lavagna:
>
> ```
> for x := 1 to 10 do begin   writeln('CIAO'); end;
> ```

Si legge: **per** x che va **da 1 a 10, fai** quello che c'è tra **begin** ed **end**. **x** è il **contatore**: una variabile **intera** (`integer`), dichiarata prima con `var x: integer;`, che a ogni giro aumenta di 1. **:=** dà il valore di partenza, **to** dice dove arriva, **do** vuol dire «fai».
Oggi lo scopo è uno: **la tua app «da … a …» funziona** e la sai spiegare. Giovedì l'hanno finita in 7 su 20: oggi tutti.

2. Quanti giri, e il numero nel Memo

```
for giro := 7 to 12 do begin   Memo1.Lines.Add(IntToStr(giro)); e
```

I giri si contano **fine − inizio + 1**: 12 − 7 + 1 = **6** giri, e nel Memo compaiono 7, 8, 9, 10, 11, 12, uno per riga.
In un programma con il form, **writeln** non si vede: si scrive nel **Memo** con **Memo1.Lines.Add** («al Memo1, alle sue righe, aggiungi»). Il Memo vuole **testo**, quindi il numero del giro passa da **IntToStr** (Int = intero, Str = testo). Per attaccare un testo al numero si usa il **+**: `IntToStr(giro) + ' - CIAO'`.

3. Le caselle: StrToInt e Memo1.Clear

Nell'app «da … a …» i numeri di partenza e di arrivo li scrive chi usa il programma, in due caselle **TEdit**. Il testo della casella diventa numero con **StrToInt** (il contrario di IntToStr):

```
inizio := StrToInt(Edit1.Text); fine := StrToInt(Edit2.Text);
```

Memo1.Clear svuota il Memo prima di ricominciare, così a ogni clic non si accumulano le righe vecchie.
Quando i numeri sono tanti (da 1 a 100) non ci stanno: con **ScrollBars = ssBoth** nel Memo compaiono le **barre di scorrimento**.

4. Gli errori di giovedì (sono normali, ma oggi li evitiamo)

1) **Identifier not found** sul contatore: manca `var giro: integer;` sopra il begin del bottone.
2) Il **punto e virgola dopo do**: `for giro := 1 to 10 do;` ripete 10 volte…

niente.
3) **begin** senza **end;**: non compila. Conta i begin e gli end: devono essere pari.
4) `Memo1.Lines.Add(giro)` senza **IntToStr**: errore, il Memo vuole testo.
5) Casella vuota e premi il bottone: StrToInt di niente dà errore. Scrivi prima i due numeri.
6) I **rientri** (regola del prof): begin ed end sullo stesso rientro, quello che sta dentro 2 spazi più a destra. In Lazarus selezioni le righe e premi **Ctrl+Shift+I** (a destra) o **Ctrl+Shift+U** (a sinistra). Codice tutto a sinistra = codice che nessuno legge: in un'azienda non passa.

5. Il programma intero (quello che oggi fai girare)

L'interfaccia: in alto **da** [5] **a** [12], al centro un Memo grande, in basso un bottone **Conta!**. Il codice del bottone, con i rientri giusti:

```
var giro, inizio, fine: integer; begin   inizio := StrToInt(Edit1
```

Leggilo riga per riga: dichiaro tre variabili intere; prendo i due numeri dalle caselle; pulisco il Memo; per giro da inizio a fine, aggiungo al Memo il numero del giro come testo. Fine.

6. LA PRATICA, passi 1-5: il form

Un passo alla volta. Ogni passo è una cosa sola.
1) Apri Lazarus: **Progetto ⟶ Nuovo progetto ⟶ Applicazione ⟶ Ok**. (Se hai il progetto di giovedì, aprilo e vai al passo 7.)
2) Dalla scheda **Standard** metti sul form un **TLabel**; nell'Ispettore oggetti la sua **Caption** diventa **da**.
3) Metti accanto un **TEdit** (Edit1) e cancella il contenuto della proprietà **Text**.
4) Metti un altro **TLabel** con Caption **a**, e accanto un altro **TEdit** (Edit2) con Text vuoto.
5) Metti al centro un **TMemo** grande; nell'Ispettore metti **ScrollBars** a **ssBoth**. Metti in basso un **TButton** con Caption **Conta!**.
Rispondi alla domanda qui sotto: così il monitor sa che sei a questo punto.

7. LA PRATICA, passi 6-9: il codice e la prova

6) Fai **doppio clic** sul bottone: si apre `procedure TForm1.Button1Click`.
7) Scrivi il codice della scheda 5, riga per riga, **con i rientri di 2 spazi**. La riga `var giro, inizio, fine: integer;` va **sopra** il begin della procedura.
8) Premi **F9**. Se Lazarus segna un errore in rosso, leggi la riga: quasi sempre è uno dei 6 errori della scheda 4.
9) Scrivi 5 e 12 nelle caselle, premi **Conta!**: compaiono i numeri da 5 a 12. **FATTO!** Prova anche da 1 a 100 e usa la barra di scorrimento.
Rispondi alla domanda qui sotto.

8. LA PRATICA, passi 10-12: fallo tuo, e la sfida

10) **Fallo tuo**: accanto al numero metti una frase tua, per esempio `IntToStr(giro) + ' - forza!'`; cambia la **Caption del form** (il titolo della finestra) con un nome tuo.
11) **Tieni pronto il codice per la verifica**: nella finestra del codice seleziona dalla riga `var` fino all'ultimo `end;` del bottone e premi **Ctrl+C**. Nella verifica c'è una domanda dove lo incolli (lì l'incolla è permesso; nelle altre no).
12) **Sfida** per chi ha finito: una terza casella con un numero, e nel Memo la sua tabellina: `IntToStr(giro) + ' x 7 = ' + IntToStr(giro * 7)`, con il 7 preso dalla terza casella. Sfida 2: un bottone **Pulisci** che fa solo `Memo1.Clear;`.
Ti blocchi? Scrivi la domanda all'assistente, in fondo alla pagina: ti risponde.

L16 20/10/2026 — Il ciclo WHILE: lancia il dado finché...

Libro parziale: lezioni/2026-10-20_ciclo-while.md

Che cosa abbiamo fatto

1. **Quando:** 20/10/2026, 2 ore, Laboratorio.
2. **Argomenti:** Ciclo WHILE in Lazarus («ripeti finché»): teoria, ripasso, gioco del dado (lancia finché esce il numero magico e conta i tiri), ciclo infinito e come fermarlo; verifica.

La teoria e l’esercitazione

Le schede della lezione

1. Dal FOR al WHILE: quando non sai quante volte

Con il **for** sai prima quanti giri fai: `for x := 1 to 10 do` fa 10 giri, sempre.
Ma se lanci un dado finché esce il 6, **quante volte lo lanci?** Non lo sa nessuno: magari 1, magari 15.
Per questo c'è il **while**: «**finché** la condizione è vera, ripeti». Si ferma quando la condizione diventa falsa.
Nella vita: *finché* ho fame, mangio; *finché* il semaforo è rosso, aspetto.

2. Come si scrive

```
while condizione do begin   istruzioni da ripetere; end;
```

Si legge: **finché** la condizione è vera, **fai** quello che c'è tra begin ed end.
La condizione si controlla **prima** di ogni giro. Se è falsa già all'inizio, il while non fa **nessun** giro.
Le condizioni sono quelle dell'IF: `=` uguale, `<>` diverso, `<` minore, `>` maggiore, `<=`, `>=`.
Esempio: conta da 1 a 5 con il while:

```
x := 1; while x <= 5 do begin   Memo1.Lines.Add(IntToStr(x));   x
```

Nel for il contatore aumenta da solo; nel while **lo fai aumentare tu** (`x := x + 1`).

3. Il pericolo: il ciclo infinito

Se dentro il while niente cambia la condizione, il ciclo **non finisce mai**:

```
x := 1; while x <= 5 do begin   Memo1.Lines.Add(IntToStr(x)); end
```

Qui manca `x := x + 1`: x resta 1 per sempre e il programma si **blocca**.
Se ti succede: in Lazarus menu **Esegui ⟶ Ferma** (oppure **Ctrl+F2**). Succede a tutti i programmatori, anche ai pro: si ferma e si corregge.
Regola: dentro il while deve sempre esserci qualcosa che **prima o poi** rende falsa la condizione.

4. Il dado: Random

Per tirare un dado il computer usa i **numeri a caso**:

```
Randomize; dado := Random(6) + 1;
```

Randomize «mescola» i numeri: si scrive una volta all'inizio, altrimenti esce sempre la stessa sequenza.
Random(6) dà un numero da **0 a 5**; con **+ 1** diventa da **1 a 6**, come un dado vero.
Per una moneta: `Random(2)` dà 0 o 1 (testa o croce).

5. Il programma: lancia finché esce il numero magico

L'interfaccia: in alto **Numero magico (1-6)** [6], al centro un Memo grande, in basso un bottone **Lancia!**. Il codice del bottone, con i rientri giusti:

```
var dado, magico, tiri: integer; begin   Randomize;   magico := S
```

Leggilo: prendo il numero magico dalla casella; tiri parte da 0; **finché** il dado è **diverso** dal numero magico, tiro il dado, conto il tiro e lo scrivo nel Memo. Quando esce, scrivo dopo quanti tiri.
Perché `dado := 0` prima del while? Perché all'inizio il dado non è ancora uscito: 0 è diverso da qualsiasi numero magico, quindi il primo giro si fa sempre.

6. LA PRATICA, passi 1-5: il form

Un passo alla volta.
1) Apri Lazarus: **Progetto ⟶ Nuovo progetto ⟶ Applicazione ⟶ Ok**.
2) Metti un **TLabel** con Caption **Numero magico (1-6)**.
3) Accanto un **TEdit** (Edit1): nella proprietà **Text** scrivi **6**.
4) Al centro un **TMemo** grande con **ScrollBars = ssBoth**; cancella le righe di **Lines**.
5) In basso un **TButton** con Caption **Lancia!**.

7. LA PRATICA, passi 6-9: il codice e la prova

6) **Doppio clic** sul bottone: si apre `procedure TForm1.Button1Click`.
7) Scrivi il codice della scheda 5, riga per riga, con i rientri di 2 spazi. La riga

`var dado, magico, tiri: integer;` va **sopra** il begin.
8) Premi **F9**.
9) Premi **Lancia!** più volte: ogni volta i tiri sono diversi. **FATTO!**
Prova: scrivi **9** nella casella e premi Lancia!. Che cosa succede? (Scheda 3: Esegui ⟶ Ferma.)

8. Fallo tuo, gli errori e la sfida

Fallo tuo: cambia le frasi nel Memo (per esempio `'Ce l''hai fatta in '`), il titolo della finestra (Caption del form) e il colore del form.
Tieni pronto il codice per la verifica: seleziona dalla riga `var` all'ultimo `end;`, **Ctrl+C**; nella verifica c'è una domanda dove lo incolli.
Errori tipici: manca `Randomize` (escono sempre gli stessi tiri); manca `tiri := tiri + 1` (dice sempre 0 tiri); `while dado = magico` invece di `<>` (non fa nessun giro); numero magico 9 (ciclo infinito).
Sfida: due dadi, finché escono **due 6** insieme (`while (d1 <> 6) or (d2 <> 6) do`). Quanti tiri servono?

L17 21/10/2026 — Sicurezza 4: diritti, doveri e vigilanza

Libro parziale: lezioni/2026-10-21_sicurezza-4.md

Che cosa abbiamo fatto

1. **Quando:** 21/10/2026, 1 ora, Sicurezza.
2. **Argomenti:** Sicurezza sul lavoro (Regione), 4ª ora: diritti, doveri, sanzioni e organi di vigilanza; quiz di 20 casi su Moduli; recupero dei quiz delle ore 1-3.

La teoria e l'esercitazione

Le schede della lezione

1. La squadra della prevenzione

La teoria della **3ª ora** è in questa pagina: leggi le 4 schede, una alla volta.

In azienda la sicurezza è organizzata come una squadra. Il **datore di lavoro** decide e risponde. Il **Servizio di Prevenzione e Protezione** (SPP), guidato dal

RSPP, studia i rischi e propone le soluzioni. Il **medico competente** controlla la salute. Il **RLS** parla a nome dei lavoratori. I **preposti** controllano sul campo.

2. Il DVR: la mappa dei rischi

Il **DVR** (Documento di Valutazione dei Rischi) è il documento dove l'azienda scrive **tutti i rischi** di ogni lavoro e **che cosa fa** per ridurli. Lo firma il datore di lavoro, con l'RSPP e il medico competente, sentito l'RLS. Il datore di lavoro **non può delegarlo** a nessuno. Si aggiorna quando cambia qualcosa: una macchina nuova, un reparto nuovo, dopo un infortunio. Almeno **una volta l'anno** c'è la **riunione periodica** della squadra della sicurezza.

3. Le emergenze: chi interviene

Ogni azienda ha gli **addetti alle emergenze**: gli **addetti antincendio** (usano gli estintori e guidano l'evacuazione) e gli **addetti al primo soccorso** (aiutano chi si fa male e chiamano il **112**). C'è un **piano di emergenza**: vie di fuga, **uscite di emergenza** con il cartello verde, **punto di raccolta** fuori dall'edificio. Durante l'evacuazione: niente ascensore, niente corsa, non si torna indietro a prendere le cose.

4. Formazione, informazione, addestramento

Informazione: ti dicono quali rischi ci sono e chi sono le persone della sicurezza. **Formazione**: un corso per capire (come questo: 4 ore generali + le ore specifiche). **Addestramento**: provi davvero, con le mani, l'attrezzo o il DPI, con qualcuno che ti guarda. Si fanno **durante l'orario di lavoro** e **non costano niente al lavoratore**. Valgono anche per lo **studente in stage**.

L18 22/10/2026 — Il salvadanaio: FOR e WHILE; GitHub in ordine

Libro parziale: lezioni/2026-10-22_salvadanaio-github.md

Che cosa abbiamo fatto

1. **Quando:** 22/10/2026, 3 ore, Laboratorio.

2. Argomenti: Interrogazioni sul ciclo WHILE; FOR e WHILE nel programma «Salvadanaio»; GitHub: il repository «lazarus» con un progetto per cartella.

La teoria e l'esercitazione

Le schede della lezione

1. Il mio salvadanaio: che cosa facciamo oggi

Oggi fai un programma **tuo**: un salvadanaio.
Tu scegli **che cosa vuoi comprare** (delle scarpe, una cuffia, un gioco) e **quanto costa**.
Ogni giorno metti nel salvadanaio da **1 a 5 euro**, a caso.
Il programma ha **tre bottoni**:
1) **Una settimana**: quanto risparmi in 7 giorni. Qui usi il **FOR**.
2) **Quanti giorni mi servono?**: quanti giorni servono per arrivare al prezzo. Qui usi il **WHILE**.
3) **Con la paghetta** (sfida): il sabato arrivano 10 euro in più.
Un pezzo alla volta. Ogni pezzo finisce con un **FATTO!** verde.

2. FOR o WHILE? Ripasso lampo

FOR = so **prima** quante volte. Una settimana ha sempre 7 giorni:

```
for giorno := 1 to 7 do
```

WHILE = **non so** quante volte. Ripeto **finché** la condizione è vera:

```
while totale < prezzo do
```

Si legge: **finché** il totale è **minore** del prezzo, metto altri soldi.
Nel FOR il contatore aumenta **da solo**. Nel WHILE lo fai aumentare **tu**: `giorni := giorni + 1`.
Nel WHILE qualcosa deve cambiare la condizione, se no il ciclo **non finisce mai** (ciclo infinito: **Esegui ⟶ Ferma**, oppure **Ctrl+F2**).

3. PEZZO 1: la Form (circa 15 minuti)

1) Apri Lazarus: **Progetto ⟶ Nuovo progetto ⟶ Applicazione ⟶ Ok**.
2) Metti un **TLabel**. Caption: **Che cosa vuoi comprare?**
3) Accanto metti un **TEdit** (si chiama **Edit1**). Nella proprietà **Text** scrivi la cosa che vuoi tu, per esempio **scarpe**.
4) Sotto metti un altro **TLabel**. Caption: **Quanto costa (euro)?**
5) Accanto metti un **TEdit** (**Edit2**). Text: il prezzo, per esempio **80**. Solo il numero, senza €.
6) Metti tre **TButton**, uno sotto l'altro. Caption: **Una settimana** (Button1), **Quanti giorni mi servono?** (Button2), **Con la paghetta** (Button3).
7) Metti un **TMemo** grande (Memo1). **ScrollBars = ssVertical**. Cancella le righe di **Lines**.
8) **Fallo tuo**: Caption del form = **Il salvadanaio di** e il tuo nome. Cambia il colore (Color) del form.
9) Premi **F9**: si apre la tua finestra.

FATTO! La finestra si apre e vedi le due caselle, i tre bottoni e il Memo.

4. PEZZO 2: il bottone «Una settimana», con il FOR (circa 20 minuti)

1) Chiudi la finestra. Fai **doppio clic** sul bottone **Una settimana**.
2) Lazarus scrive la procedura. Completala così: le righe con `// QUI` dicono che cosa scrivere al loro posto.

```
procedure TForm1.Button1Click(Sender: TObject); var giorno, euro,
```

Indizi, se ti servono:
Il ciclo: `for giorno := 1 to 7 do` poi `begin` … `end;`
Il soldo del giorno: `euro := Random(5) + 1;` (da 1 a 5)
Nel salvadanaio: `totale := totale + euro;`
La riga nel Memo: `Memo1.Lines.Add('Giorno ' + IntToStr(giorno) + ': +' + IntToStr(euro) + ' euro');`
Il totale, **dopo** l'`end;` del ciclo: `Memo1.Lines.Add('In una settimana: ' + IntToStr(totale) + ' euro');`
3) Premi **F9** e poi il bottone **Una settimana**, più volte.

FATTO! Nel Memo vedi 7 righe, da Giorno 1 a Giorno 7, e sotto il totale. Ogni volta i numeri cambiano.

5. PEZZO 3: il bottone «Quanti giorni mi servono?», con il WHILE (circa 25 minuti)

1) Doppio clic sul bottone **Quanti giorni mi servono?** e completa:

```
procedure TForm1.Button2Click(Sender: TObject); var giorni, euro,
```

Indizi:
Il ciclo: `while totale < prezzo do` poi `begin` … `end;`
Dentro: `euro := Random(5) + 1;` poi `totale := totale + euro;` poi `giorni := giorni + 1;`
Se vuoi vedere i giorni nel Memo: `Memo1.Lines.Add('Giorno ' + IntToStr(giorni) + ': ho ' + IntToStr(totale) + ' euro');`
Dopo il ciclo: `Memo1.Lines.Add('Per ' + Edit1.Text + ' ti servono ' + IntToStr(giorni) + ' giorni');`
Perché si ferma? Ogni giro il totale cresce. Quando arriva al prezzo (o lo supera), `totale < prezzo` diventa **falso** e il ciclo finisce. Questa è la **condizione di uscita**.
2) F9, scrivi 80 nella casella del prezzo, premi il bottone.

FATTO! Il Memo dice: Per scarpe ti servono … giorni (con la tua cosa e il tuo prezzo).

6. PEZZO 4: quante settimane? (circa 10 minuti)

In Pascal ci sono due operazioni per le divisioni **senza virgola**:
div = quante volte ci sta. `23 div 7` = **3** (3 settimane intere).
mod = quanto avanza. `23 mod 7` = **2** (avanzano 2 giorni).
Aggiungi alla fine del bottone **Quanti giorni mi servono?**, dopo l'altra riga:

```
Memo1.Lines.Add('Cioè ' + IntToStr(giorni div 7) + ' settimane e
```

Prova con un prezzo grande, per esempio **150**.

FATTO! Il Memo dice anche: Cioè … settimane e … giorni.

7. SFIDA: il bottone «Con la paghetta» (circa 20 minuti)

Il **sabato** arriva la paghetta: **10 euro in più**. Facciamo finta che il giorno 7, il 14, il 21… siano i sabati.
Come si riconosce? Il giorno è un sabato quando `giorni mod 7 = 0` (7, 14, 21 divisi per 7 non hanno resto).
1) Doppio clic sul bottone **Con la paghetta**.
2) Copia dentro tutto il codice del bottone «Quanti giorni mi servono?» (le var e le righe tra begin ed end).
3) Dentro il while, **dopo** `giorni := giorni + 1;`, aggiungi:

```
if giorni mod 7 = 0 then begin   totale := totale + 10;   Memo1.L
```

4) F9 e premi i due bottoni con lo stesso prezzo. Con la paghetta servono **meno giorni**?

FATTO! Nel Memo compare «Sabato: paghetta +10 euro!» e i giorni sono meno di prima.

8. Errori tipici e la consegna

Errori tipici (succede a tutti, anche ai pro):
1) **Identifier not found**: manca la variabile nella riga `var`.
2) Il totale è sempre 0 o sbagliato: manca `totale := 0;` prima del ciclo, o `totale := totale + euro;` dentro.
3) Dice sempre 0 giorni: manca `giorni := giorni + 1;`.
4) Il programma si blocca: ciclo infinito. Il totale non cresce dentro il while. **Esegui ⟶ Ferma** e controlla.
5) Errore quando premi il bottone: la casella del prezzo è vuota o ha lettere (o il simbolo €). Scrivi solo il numero.
6) Il totale della settimana compare 7 volte: la riga del totale è **dentro** il ciclo. Spostala dopo l'`end;`.
La consegna: nella verifica c'è una domanda dove incolli **tutto il tuo codice**. In Lazarus, nella finestra del codice, seleziona dalla prima `procedure`

`TForm1.Button1Click` fino all'ultimo `end;` dei bottoni e premi **Ctrl+C**. Poi nella domanda premi **Ctrl+V**. Poi scrivi con parole tue che cosa fa.

FATTO! Hai il codice copiato e sai dire a voce che cosa fa ogni bottone.

Le schede della lezione

1. Che cosa facciamo oggi: tutti uguali, tutto in ordine

Oggi metti in ordine i tuoi progetti, come fa un programmatore vero.
Un datore di lavoro guarda il tuo GitHub: deve trovare **ogni progetto nella sua cartella**, non file sparsi.
La regola è una sola: **una cartella grande «lazarus»** e dentro **una cartella per ogni progetto**.
Sul tuo PC e su GitHub l'albero è **uguale**:

```
lazarus ├── primo-programma ├── salvadanaio ├── esercizio-3 ├── e
```

Le cartelle «esercizio-3» e «esercizio-4» sono per i prossimi esercizi: oggi le prepari vuote, poi il prof ti dice il nome vero.
Un passo alla volta. Ogni passo finisce con un **FATTO!** verde.

2. Le parole di oggi

Repository = la cartella grande del tuo progetto su GitHub. Il tuo si chiama **lazarus**.
Cartella (in inglese *folder*) = una scatola dentro il repository. Una per progetto.
Caricare (in inglese *upload*) = mandare i file dal PC a GitHub.
Commit = il salvataggio su GitHub, con una frase che dice che cosa hai fatto.
README = la pagina di presentazione del repository: chi sei e che cosa c'è dentro.
Il sito è in inglese: **Add file** = aggiungi file, **New Folder** = nuova cartella, **Delete** = cancella, **Move** = sposta.

3. PASSO 1 e 2: la cartella «lazarus» sul PC

1) **Chiudi Lazarus** (la X in alto a destra). Se chiede di salvare, premi **Sì**. Con Lazarus aperto le cartelle non si spostano.
2) Apri **Esplora file** (la cartella gialla in basso) e clicca **Documenti** a sinistra.
3) In alto clicca **Nuova cartella** e scrivi il nome, tutto minuscolo:

```
lazarus
```

4) Premi **Invio**.

FATTO! In Documenti c'è la cartella «lazarus».

4. PASSO 3 e 4: dentro «lazarus», le cartelle dei progetti

1) Trova la cartella del tuo **salvadanaio** (in Documenti, o dove l'hai salvata: in Lazarus il percorso era scritto in alto, tra parentesi).
2) **Trascinala** sopra la cartella **lazarus** e lascia andare: adesso è dentro.
3) Se si chiama in un altro modo (salvadanio, for e while...), tasto destro ⟶ **Rinomina** e scrivi:

```
salvadanaio
```

4) Entra in **lazarus** (doppio clic) e crea due cartelle nuove con **Nuova cartella**:

```
esercizio-3
```

```
esercizio-4
```

FATTO! Dentro Documenti\lazarus ci sono salvadanaio, esercizio-3 ed esercizio-4.

5. PASSO 5: apri il tuo repository nell'editor di GitHub

1) In Chrome apri **github.com** e il tuo repository **lazarus** (in alto a destra la tua foto ⟶ **Your repositories** ⟶ **lazarus**).
2) Clicca una volta in un punto vuoto della pagina e premi il tasto **.** (il punto).

3) Si apre **github.dev**: è come un programma, ma nel browser. A sinistra vedi i file del repository.
4) A sinistra, sotto l'ultimo file, clicca con il tasto **destro** nello spazio vuoto ⟶ **New Folder...** e scrivi:

```
primo-programma
```

e premi Invio.

FATTO! A sinistra c'è la cartella «primo-programma».

6. PASSO 6: i file sparsi vanno in «primo-programma»

I file nella pagina principale (project1, unit1, ...) sono il tuo progetto vecchio. Mettiamoli nella sua cartella.
1) A sinistra clicca il **primo file** sotto le cartelle.
2) Tieni premuto **Maiusc** e clicca l'**ultimo file**: si colorano tutti.
3) Tieni premuto **Ctrl** e clicca **README.md**: il README **non** si sposta.
4) **Trascina** i file colorati sopra la cartella **primo-programma** e clicca **Move**.
Nessun file sparso, solo il README? Questo passo è già fatto: vai al passo 7.

FATTO! A sinistra restano solo le cartelle e README.md.

7. PASSO 7: via i file .exe

Il file **.exe** è il programma già pronto: pesa tanto e non serve, perché il progetto si ricrea da Lazarus con F9.
1) Apri le cartelle a sinistra (clic sulla freccina) e cerca i file che finiscono con **.exe**.
2) Clicca il file .exe e premi **Canc**, poi **Delete**.
3) Fallo per ogni .exe. Le cartelle **lib** e **backup** puoi cancellarle allo stesso modo: le rifà Lazarus.

FATTO! Nel repository non ci sono più file .exe.

8. PASSO 8: le cartelle dei prossimi esercizi

GitHub non tiene le cartelle vuote: dentro ci mettiamo un piccolo README.
1) A sinistra, tasto destro nello spazio vuoto ⟶ **New File...** e scrivi, con la barra **/**:

```
esercizio-3/README.md
```

La barra crea la cartella e il file dentro. Premi Invio.
2) Nel file che si apre scrivi:

```
Qui andrà l'esercizio 3
```

3) Rifai per il 4:

```
esercizio-4/README.md
```

```
Qui andrà l'esercizio 4
```

FATTO! A sinistra ci sono esercizio-3 ed esercizio-4.

9. PASSO 9: il tuo README (Fallo tuo) e il commit

1) A sinistra clicca **README.md** (quello fuori dalle cartelle).
2) Cancella tutto (**Ctrl+A**, **Canc**) e scrivi la tua presentazione. Puoi partire da questo e cambiarlo come vuoi tu:

```
# I miei progetti Lazarus  Sono (il tuo nome), classe 2INF.  1. p
```

3) Nella barra delle icone tutta a sinistra clicca la **terza icona** (quella con il pallino blu e un numero).
4) Nella casella **Message** scrivi:

```
Metto in ordine i miei progetti
```

5) Clicca il bottone verde **Commit & Push**.

FATTO! Il commit è partito: sul tuo repository vedi le cartelle e il README nuovo.

10. Errori tipici (succede a tutti, anche ai pro)

1) **La cartella ha un nome sbagliato** (salvadanio, for e while, salvadanio2): in github.dev tasto destro sulla cartella ⟶ **Rename** e scrivi il nome giusto, poi commit.
2) **Ho spostato anche il README** dentro primo-programma: trascinalo fuori, sopra lo spazio vuoto in basso.
3) **Commit & Push è grigio**: non hai scritto il messaggio nella casella **Message**.
4) **Non vedo le cartelle su github.com**: ricarica la pagina (**F5**). Hai premuto Commit & Push?
5) **Ho due repository** (lazarus e lazarus-): ne tieni uno solo e si chiama **lazarus**. Chiedi all'assistente.

FATTO! Hai risolto il tuo errore da solo, o con l'assistente.

11. PASSO 10 e 11: il salvadanaio su GitHub

1) Torna su **github.com**, nel tuo repository **lazarus**, e ricarica la pagina (F5).
2) Clicca **Add file** (a sinistra del bottone verde Code) ⟶ **Upload files**.
3) Apri **Esplora file** ⟶ **Documenti** ⟶ **lazarus**.
4) **Trascina la cartella salvadanaio** dentro la pagina di GitHub (porta il mouse sull'icona di Chrome in basso e aspetta un secondo, poi lascia al centro della pagina).
5) Aspetta la barra verde piena e clicca **Commit changes**.
6) Guarda la tabella in alto in questa pagina: entro 5 minuti la tua riga diventa verde.

FATTO! Il tuo GitHub è uguale alla cartella lazarus del tuo PC. Fallo vedere al compagno!

L19 26/10/2026 — I file di un progetto Lazarus e il repository in ordine

Libro parziale: lezioni/2026-10-26_file-lazarus.md

Che cosa abbiamo fatto

1. **Quando:** 26/10/2026, 3 ore, Laboratorio.
2. **Argomenti:** A che cosa serve il repository; il repository «lazarus» in ordine, una cartella per progetto; i file di un progetto Lazarus: testo e binari, quali vanno su GitHub (.lpi .lpr .pas .lfm) e quali no (.exe, lib, backup); report finale su Classroom.

La teoria e l’esercitazione

01 Che cosa crea Lazarus

1. Quando salvi un progetto, Lazarus mette nella cartella tanti file, non uno solo.
2. Alcuni sono la RICETTA: li hai fatti tu (il codice, la finestra). Altri sono la TORTA già cotta: Lazarus li crea da solo quando premi Esegui.
3. Su GitHub mettiamo solo la ricetta. Con la ricetta chiunque rifà la torta: apre il progetto e preme Esegui.

02 File di testo e file binari

1. Un file di TESTO si apre con Blocco note e dentro leggi parole e codice.
2. Un file BINARIO aperto con Blocco note mostra solo simboli strani: lo capisce solo il computer.
3. Prova: clic destro sul file, poi «Apri con», poi «Blocco note». Se leggi parole è testo, se vedi simboli è binario. Chiudi senza salvare.

03 Tutti i file, uno per uno

Estensione	Nome	A che cosa serve	Testo o binario	Su GitHub
.lpi	Lazarus Project Information	La scheda del progetto: le sue impostazioni. Con un doppio clic si apre il progetto in Lazarus.	testo	SÌ
.lpr	Lazarus Program	Il programma principale: il punto da cui parte tutto.	testo	SÌ
.pas	Pascal (unit)	Il codice che scrivi tu, per esempio unit1.pas.	testo	SÌ
.lfm	Lazarus Form	Com'è fatta la finestra: bottoni, caselle, posizioni, Caption.	testo	SÌ
.ico	Icona	L'icona del programma. Lazarus non la rifà da solo.	binario	SÌ
.lps	Lazarus Project Session	Quali file avevi aperto e dove era il cursore. È solo tua.	testo	NO
.res	Risorse	L'icona e altre informazioni già «impacchettate». Lazarus la rifà.	binario	NO
.o .ppu	Pezzi compilati	Pezzi di programma già tradotti per il computer. Lazarus li rifà.	binario	NO
.compiled	Nota di compilazione	Un appunto di Lazarus su come ha compilato.	testo	NO
.exe	Programma pronto	Il programma finito. Lazarus lo rifà quando premi Esegui.	binario	NO
backup/	Cartella	Copie vecchie dei tuoi file, fatte da Lazarus.	—	NO
lib/	Cartella	I pezzi compilati (.o, .ppu, .res, .compiled).	—	NO

04 La regola facile

1. Su GitHub vanno solo 4 tipi di file: .lpi, .lpr, .pas, .lfm (e l'icona .ico, se l'hai scelta). Tutto il resto Lazarus lo rifà da solo.

2. Perché niente .exe? GitHub serve per conservare la ricetta. Il .exe è la torta: pesa, cambia ogni volta che premi Esegui e non si può leggere. Chi vuole il programma apre il progetto e preme Esegui.

05 Come deve essere il tuo repository

```
lazarus/
├── README.md
├── primo-programma/
│   ├── project1.lpi
│   ├── project1.lpr
│   ├── unit1.pas
│   └── unit1.lfm
└── salvadanaio/
    ├── salvadanaio.lpi
    ├── salvadanaio.lpr
    ├── unit1.pas
    └── unit1.lfm
```

06 Sai spiegarlo?

Rispondi a voce, con parole tue:

1. In quale file c’è il codice che hai scritto tu?
2. In quale file c’è com’è fatta la tua finestra?
3. Come capisci se un file è di testo o binario?
4. Perché su GitHub non mettiamo il file .exe?

L20 27/10/2026 — La calcolatrice in Lazarus e su GitHub

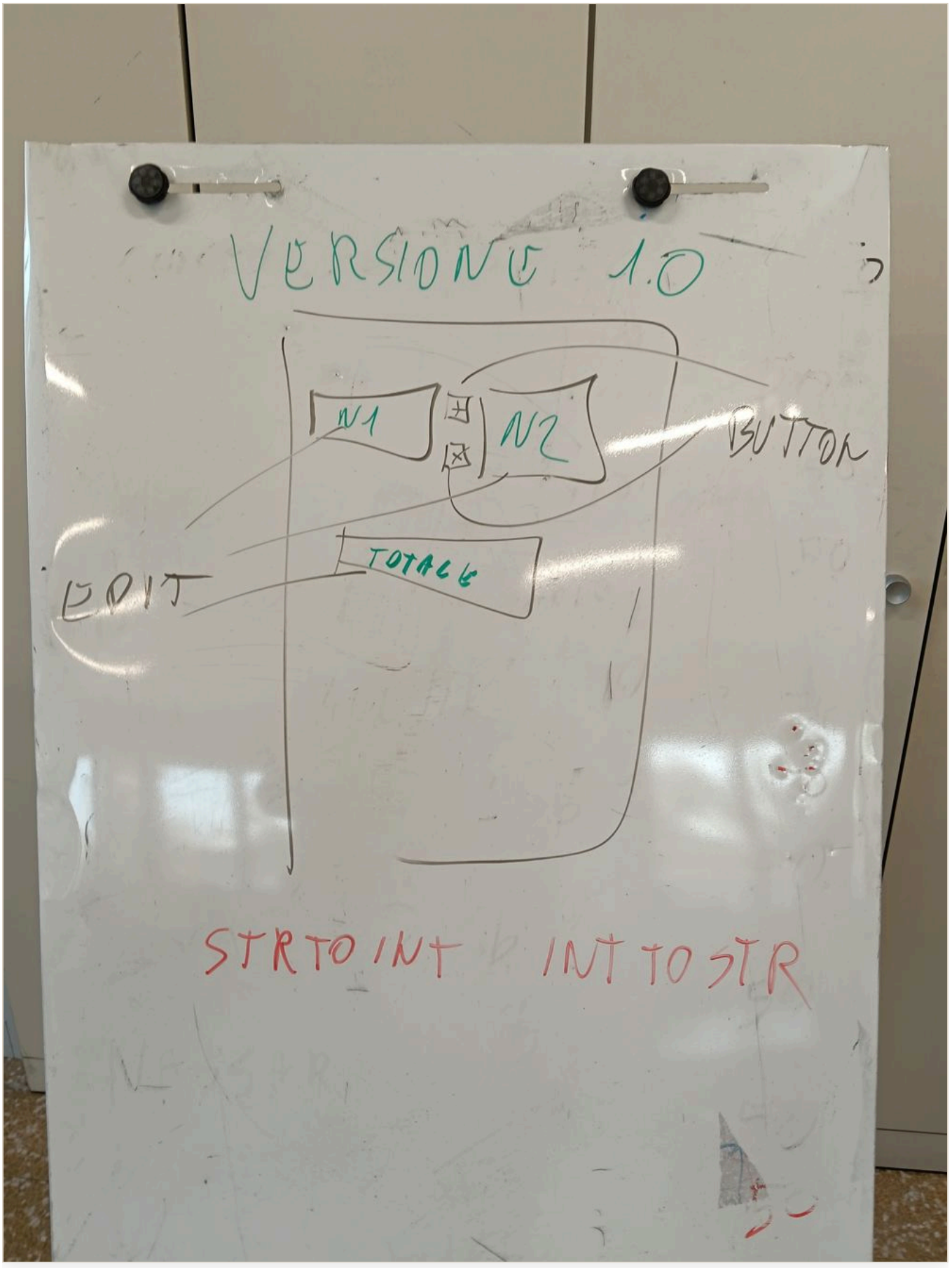
Libro parziale: lezioni/2026-10-27_calcolatrice.md

Che cosa abbiamo fatto

1. **Quando:** 27/10/2026, 2 ore, Informatica.
2. **Argomenti:** La calcolatrice in Lazarus (versione 1.0 dalla lavagna) e il suo posto su GitHub.

La teoria e l’esercitazione

01 La lavagna: la calcolatrice versione 1.0



La lavagna di oggi: la finestra della calcolatrice versione 1.0

1. La finestra (form) ha tre caselle **Edit**: N1 (primo numero), N2 (secondo numero) e TOTALE (il risultato).
2. Ha due bottoni (**Button**): **+** (più) e **×** (per).
3. Quando premi un bottone, il programma legge N1 e N2, fa il conto e scrive il risultato in TOTALE.

02 StrToInt e IntToStr: testo e numeri

1. Quello che scrivi in una casella Edit è **testo**, anche se sono cifre: «12» è come una parola.
2. Con il testo non si fanno i conti. Prima va trasformato in numero: **StrToInt** (da testo a numero intero).
3. Il risultato è un numero. Per scriverlo nella casella va ritrasformato in testo: **IntToStr** (da numero intero a testo).
4. Il giro è sempre lo stesso: testo ⟶ StrToInt ⟶ conto ⟶ IntToStr ⟶ testo.

03 Il codice della versione 1: più e per

```
procedure TForm1.ButtonPiuClick(Sender: TObject);
begin
  Edit3.Text := IntToStr(StrToInt(Edit1.Text) + StrToInt(Edit2.Text));
end;

procedure TForm1.ButtonPerClick(Sender: TObject);
begin
  Edit3.Text := IntToStr(StrToInt(Edit1.Text) * StrToInt(Edit2.Text));
end;
```

1. Edit1 è N1, Edit2 è N2, Edit3 è TOTALE.
2. Il codice di ogni bottone si scrive con un doppio clic sul bottone nel form.
3. Prova con Esegui: 7 + 5 = 12, 7 × 5 = 35.

04 La versione 2: meno e divisione intera

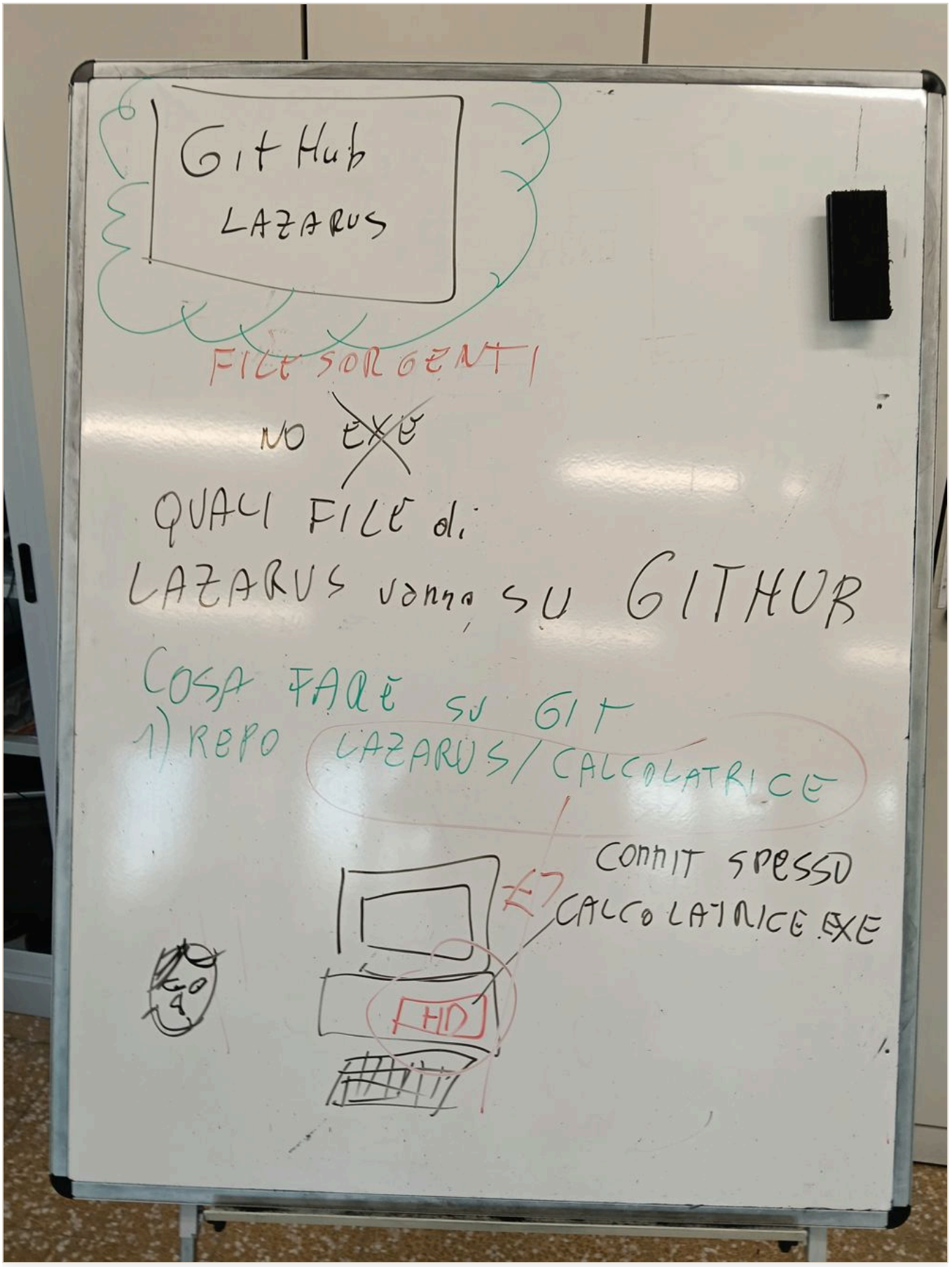
```
procedure TForm1.ButtonMenoClick(Sender: TObject);
begin
  Edit3.Text := IntToStr(StrToInt(Edit1.Text) - StrToInt(Edit2.Text));
end;

procedure TForm1.ButtonDivClick(Sender: TObject);
begin
```

```
  if StrToInt(Edit2.Text) = 0 then
    Edit3.Text := 'Non si divide per zero'
  else
    Edit3.Text := IntToStr(StrToInt(Edit1.Text) div StrToInt(Edit2.Text));
end;
```

1. **div** è la divisione intera: dà solo la parte intera, senza virgola. 7 div 2 = 3.
2. Dividere per zero non si può: il programma andrebbe in errore. Per questo prima controlliamo se N2 è 0 (if … then … else).

05 Il tuo PC e GitHub



La lavagna: su GitHub vanno i file sorgenti, niente .exe; repository lazarus, cartella calcolatrice; commit spesso

1. Quando lavori con Lazarus, il programma sta sul tuo PC, nel disco fisso (HD): lavori **in locale**.

2. GitHub è su internet: una copia del progetto che resta al sicuro anche se il PC si rompe.

3. I due posti non si aggiornano da soli: sei tu a portare i file dal PC a GitHub.

4. Su GitHub vanno solo i **file sorgenti**: .lpi, .lpr, .pas, .lfm. Il file calcolatrice.exe NO: Lazarus lo rifà quando premi Esegui.

Che cosa	Dove	Quando	Come
Salvare	sul tuo PC (in locale)	spessissimo, ogni pochi minuti	in Lazarus premi Ctrl + S
Commit	su GitHub (su internet)	ogni volta che una cosa nuova funziona	carichi i 4 file e premi «Commit changes»

```
lazarus/
├── README.md
├── primo-programma/
├── salvadanaio/
└── calcolatrice/
    ├── calcolatrice.lpi
    ├── calcolatrice.lpr
    ├── unit1.pas
    └── unit1.lfm
```

06 Su GitHub, passo per passo

1. Apri la pagina guidata: La calcolatrice su GitHub. Ogni passo si apre solo quando quello prima è giusto.

2. La prima volta: crea la cartella con «Add file», «Create new file», nome **calcolatrice/README.md**, «Commit changes».

3. Poi entra nella cartella calcolatrice: «Add file», «Upload files», «choose your files», scegli SOLO i 4 file con Ctrl, «Commit changes».

4. Ogni volta che una cosa nuova funziona: Ctrl + S sul PC, poi ricarichi i 4 file su GitHub con un messaggio che dice che cosa hai cambiato (es. «aggiunto il bottone per»).

07 La release: v1.0 e v2.0

1. Una **release** è una versione finita e funzionante, con un numero.
2. Versione 1 (più e per): nel repository lazarus, a destra, «Releases», «Create a new release», «Choose a tag», scrivi **v1.0**, «Create new tag», titolo «Calcolatrice versione 1», «Publish release».
3. Versione 2 (anche meno e divisione intera): lo stesso con **v2.0**.

08 Sai spiegarlo?

1. A che cosa servono StrToInt e IntToStr?
2. Che differenza c'è tra salvare con Ctrl + S e fare un commit?
3. Perché su GitHub non mettiamo il file .exe?
4. Che cos'è la release v1.0?

L21 28/10/2026 — Sicurezza: ripasso delle ore 1-4

Libro parziale: lezioni/2026-10-28_sicurezza-ripasso.md

Che cosa abbiamo fatto

1. **Quando:** 28/10/2026, 1 ora, Sicurezza.
2. **Argomenti:** Ripasso delle ore 1-4 della formazione generale e recupero dei quiz mancanti su Moduli.

La teoria e l'esercitazione

2INF · 28/10/2026 · ultima ora della formazione generale

Usa SOLO i quiz di questa pagina (Google Moduli). NON fare il TEST sul sito della Regione: si fa a gennaio-febbraio.

1. Ripasso delle 4 ore

Leggi le 4 schede qui sotto: il ripasso è tutto in questa pagina.

1ª ora: rischio e danno

Il **pericolo** è una cosa che può fare male: un filo scoperto, un pavimento bagnato.

Il **danno** è il male che succede davvero: una ferita, una malattia.

Il **rischio** è quanto è facile che il danno succeda e quanto è grave: rischio = probabilità × danno.

2ª ora: prevenzione e protezione

La **prevenzione** agisce prima: fa in modo che il danno succeda meno spesso (formazione, ordine, regole).

La **protezione** riduce il danno se succede: protezioni sulle macchine e i **DPI** (dispositivi di protezione individuale: guanti, occhiali, scarpe).

3ª ora: organizzazione della prevenzione

Il **datore di lavoro** valuta i rischi e scrive il **DVR** (documento di valutazione dei rischi).

Lo aiutano l'**RSPP** (responsabile del servizio di prevenzione e protezione), il **medico competente** e il **preposto**, che controlla il lavoro ogni giorno.

I lavoratori scelgono l'**RLS** (rappresentante dei lavoratori per la sicurezza).

4ª ora: diritti, doveri, sanzioni e vigilanza

Il lavoratore ha **diritto** alla formazione, ai DPI e alla visita medica.

Ha anche dei **doveri**: usare i DPI, seguire le regole, segnalare subito un pericolo.

Chi non rispetta le regole riceve una **sanzione** (una multa o una punizione). Gli **organi di vigilanza** (ATS e Ispettorato del lavoro) fanno i controlli.

2. I TUOI quiz da fare

Passo 1. Scrivi il tuo cognome e il tuo nome, come sul registro.

Cognome e nome

Passo 2. Premi il bottone.

Il Modulo si apre con l'account della scuola. Dopo l'invio, la pagina si aggiorna entro qualche minuto.

2 Changelog

1. **v1.0 (29/10/2026)**: prima versione completa: le 21 lezioni dal 21/09 al 28/10, ognuna con la teoria e l'esercitazione (prende il posto della dispensa v0.1).